МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий
Кафедра вычислительной техники

КУРСОВАЯ РАБОТА

По дисциплине «Объектно-ориентированное программирование»
(наименование дисциплины)

Тема курсовой работы К_5 Моделирование работы кофе машины
(наименование темы)

Студент группы ИКБО-63-24 Мишакин Глеб Святославович
(учебная группа) *(Фамилия Имя Отчество)* *(подпись студента)*

Руководитель курсовой работы доцент ~~Лозовский В.В.~~ Грач Е.П.
(Должность, звание, ученая степень) *(подпись руководителя)*

Консультант ст.преп. Грач Е.П.
(Должность, звание, ученая степень) *(подпись консультанта)*

Работа представлена к защите «31» мая 2025 г.

Допущен к защите «31» мая 2025 г.

ХОР не выполнен п. 5

Москва 2025 г.

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий
Кафедра вычислительной техники

Утверждаю

Заведующий кафедрой _____
Подпись

Платонова О.В.
ФИО

«24» февраля 2025г.

ЗАДАНИЕ
На выполнение курсовой работы
по дисциплине **«Объектно-ориентированное программирование»**

Студент Мишакин Глеб Святославович Группа ИКБО-63-24

Тема К_5 Моделирование работы кофе машины

Исходные данные:

1. Описания исходной иерархии дерева объектов.
2. Описание схемы взаимодействия объектов.
3. Множество команд для управления функционированием моделируемой системы.

Перечень вопросов, подлежащих разработке, и обязательного графического материала:

1. Построение версий программ.
2. Построение и работа с деревом иерархии объектов.
3. Взаимодействия объектов посредством интерфейса сигналов и обработчиков.
4. Блок-схемы алгоритмов.
5. Управление функционированием моделируемой системы

Срок представления к защите курсовой работы: до «31» мая 2025 г.

Задание на курсовую работу выдал _____ (Лозовский В.В.)
Подпись *ФИО консультанта*

«24» февраля 2025 г.

Задание на курсовую работу получил _____ (Мишакин Г.С.)
Подпись *ФИО исполнителя*

«24» февраля 2025 г.

Москва 2025 г.

ОТЗЫВ

на курсовую работу

по дисциплине «Объектно-ориентированное программирование»

Студент **Мишакин Глеб Святославович** группа ИКБО-63-24

(ФИО студента) *(Группа)*

Характеристика курсовой работы

Критерий	Да	Нет	Не полностью
1. Соответствие содержания курсовой работы указанной теме	+		
2. Соответствие курсовой работы заданию	+		
3. Соответствие рекомендациям по оформлению текста, таблиц, рисунков и пр.	+		
4. Полнота выполнения всех пунктов задания			+
5. Логичность и системность содержания курсовой работы	+		
6. Отсутствие фактических грубых ошибок	+		

Замечаний: не выполнен п.5

Рекомендуемая оценка: хор

Грич Е.П.

доцент Лозовский В.В.

(Подпись руководителя) *(ФИО руководителя)*

СОДЕРЖАНИЕ

ВВЕДЕНИЕ……..5

1 Постановка задачи……7

2 Метод решения……...13

3 Описание Алгоритмов…….....16

4 Блок-схемы алгоритмов…….....31

ЗАКЛЮЧЕНИЕ……56

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ……...57

Приложение 1. код программы……...58

Приложени 2. тестирование…….....74

ВВЕДЕНИЕ

Настоящая курсовая работа выполнена в соответствии с требованиями ГОСТ Единой системы программной документации (ЕСПД) [1]. Все этапы решения задач курсовой работы фиксированы, соответствуют требованиям, приведенным в методическом пособии для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [2-3] и методике разработки объектно-ориентированных программ [4-6].

Объектно-ориентированное программирование- это парадигма разработки программирования, в которой основной концепцией является понятие объекта, который отождествляется с предметной областью. Объект - отдельный представитель класса, имеющий конкретное поведение и состояние, полностью определяемое классом. Объект имеет конкретные значения атрибутов и методы, которые работают с этими значениями на основе заданнных в классе правил.[1] Появление ООП обусловливалось необходимостью решения возникающих проблем при построении сложных систем. ООП позволяет использовать один и тот же программный код с разными данными. На основе классов создаётся множество объектов, у каждого из которых могут быть собственные значения полей. Нет необходимости вводить множество переменных. Наследование позволяет не писать новый код. Основные принципы ООП: Абстракция - набор значимых характеристик объекта, исключая из рассмотрения незначимых. Полиморфизм - свойство системы использовать объекты с одинаковым интерфейсом без информации о типе внутренней структуре объекта. Наследование - свойство системы, которое позволяет описать новый класс на основе уже существующего с частично или полностью заимствующейся функциональностью. Инкапсуляция - свойство системы, позволяющее объединить

данные и метода, работающие с ними , в классе и скрыть детали реализации от пользователя.[2]

1 ПОСТАНОВКА ЗАДАЧИ

Реализовать механизм взаимодействия объектов с использованием сигналов и обработчиков, с передачей вместе сигналом текстового сообщения (строковой переменной).

Для организации взаимосвязи по механизму сигналов и обработчиков в базовый класс добавить три метода:

- установления связи между сигналом текущего объекта и обработчиком целевого объекта;
- удаления (разрыва) связи между сигналом текущего объекта и обработчиком целевого объекта;
- выдачи сигнала от текущего объекта с передачей строковой переменной.

Включенный объект может выдать или обработать сигнал.

Методу установки связи передать указатель на метод сигнала текущего объекта, указатель на целевой объект и указатель на метод обработчика целевого объекта.

Методу удаления (разрыва) связи передать указатель на метод сигнала текущего объекта, указатель на целевой объект и указатель на метод обработчика целевого объекта.

Методу выдачи сигнала передать указатель на метод сигнала и строковую переменную. В данном методе реализовать алгоритм:

1. Если текущий объект отключен, то выход, иначе к пункту 2.
2. Вызов метода сигнала с передачей строковой переменной по ссылке.
3. Цикл по всем связям сигнал-обработчик текущего объекта:

    3.1. Если в очередной связи сигнал-обработчик участвует метод сигнала, переданный по параметру, то проверить готовность целевого объекта. Если целевой объект готов, то вызвать метод обработчика

целевого объекта указанной в связи и передать в качестве аргумента строковую переменную по значению.

4. Конец цикла.

Для приведения указателя на метод сигнала и на метод обработчика использовать параметризированное макроопределение препроцессора.

В базовый класс добавить метод определения абсолютной пути до текущего объекта. Этот метод возвращает абсолютный путь текущего объекта.

Состав и иерархия объектов строится посредством ввода исходных данных. Ввод организован как в версии № 3 курсовой работы. Если при построении дерева иерархии возникает ситуация дубляжа имен среди починенных у текущего головного объекта, то новый объект не создается.

Система содержит объекты шести классов с номерами: 1, 2, 3, 4, 5, 6. Классу корневого объекта соответствует номер 1. В каждом производном классе реализовать один метод сигнала и один метод обработчика.

Каждый метод сигнала с новой строки выводит:

```
Signal from «абсолютная координата объекта»
```

Каждый метод сигнала добавляет переданной по параметру строке текста номер класса принадлежности текущего объекта по форме:

```
«пробел»(class: «номер класса»)
```

Каждый метод обработчика с новой строки выводит:

```
Signal to «абсолютная координата объекта»   Text: «переданная строка»
```

Моделировать работу системы, которая выполняет следующие команды с параметрами:

- EMIT «координата объекта» «текст» – выдает сигнал от заданного по координате объекта;
- SET_CONNECT «координата объекта выдающего сигнал» «координата

целевого объекта» – устанавливает связь;

- DELETE_CONNECT «координата объекта выдающего сигнал» «координата целевого объекта» – удаляет связь;
- SET_CONDITION «координата объекта» «значение состояния» – устанавливает состояние объекта.
- END – завершает функционирование системы (выполнение программы).

Реализовать алгоритм работы системы:

- в методе построения системы:
    - построение дерева иерархии объектов согласно вводу;
    - ввод и построение множества связей сигнал-обработчик для заданных пар объектов.
- в методе отработки системы:
    - привести все объекты в состоянии готовности;
    - цикл до признака завершения ввода:
        - ввод наименования объекта и текста сообщения;
        - вызов сигнала заданного объекта и передача в качестве аргумента строковой переменной, содержащей текст сообщения.
    - конец цикла.

Допускаем, что все входные данные вводятся синтаксически корректно. Контроль корректности входных данных можно реализовать для самоконтроля работы программы. Не оговоренные, но необходимые функции и элементы классов добавляются разработчиком.

1.1 Описание входных данных

В методе построения системы.

Множество объектов, их характеристики и расположение на дереве

иерархии. Структура данных для ввода согласно изложенному в версии № 3 курсовой работы.

После ввода состава дерева иерархии построчно вводится:

```
«координата объекта выдающего сигнал» «координата целевого объекта»
```

Ввод информации для построения связей завершается строкой, которая содержит:

```
«end_of_connections»
```

В методе запуска (отработки) системы построчно вводятся множество команд в производном порядке:

- EMIT «координата объекта» «текст» – выдать сигнал от заданного по координате объекта;
- SET_CONNECT «координата объекта выдающего сигнал» «координата целевого объекта» – установка связи;
- DELETE_CONNECT «координата объекта выдающего сигнал» «координата целевого объекта» – удаление связи;
- SET_CONDITION «координата объекта» «значение состояния» – установка состояния объекта.
- END – завершить функционирование системы (выполнение программы).

Команда END присутствует обязательно.

Если координата объекта задана некорректно, то соответствующая операция не выполняется и с новой строки выдается сообщение об ошибке.

Если не найден объект по координате:

```
Object «координата объекта» not found
```

Если не найден целевой объект по координате:

```
Handler object «координата целевого объекта» not found
```

Пример ввода:

```
appls_root
/ object_s1 3
/ object_s2 2
/object_s2 object_s4 4
/ object_s13 5
/object_s2 object_s6 6
/object_s1 object_s7 2
endtree
/object_s2/object_s4 /object_s2/object_s6
/object_s2 /object_s1/object_s7
/ /object_s2/object_s4
/object_s2/object_s4 /
end_of_connections
EMIT /object_s2/object_s4 Send message 1
EMIT /object_s2/object_s4 Send message 2
EMIT /object_s2/object_s4 Send message 3
EMIT /object_s1 Send message 4
END
```

1.2 Описание выходных данных

Первая строка:

```
Object tree
```

Со второй строки вывести иерархию построенного дерева.

Далее, построчно, если отработал метод сигнала:

```
Signal from «абсолютная координата объекта»
```

Если отработал метод обработчика:

```
Signal to «абсолютная координата объекта»   Text: «переданная строка»
```

Пример вывода:

```
Object tree
appls_root
    object_s1
        object_s7
    object_s2
        object_s4
        object_s6
    object_s13
Signal from /object_s2/object_s4
Signal to /object_s2/object_s6 Text:  Send message 1 (class: 4)
Signal to / Text:  Send message 1 (class: 4)
Signal from /object_s2/object_s4
```

```
Signal to /object_s2/object_s6 Text:  Send message 2 (class: 4)
Signal to / Text:  Send message 2 (class: 4)
Signal from /object_s2/object_s4
Signal to /object_s2/object_s6 Text:  Send message 3 (class: 4)
Signal to / Text:  Send message 3 (class: 4)
Signal from /object_s1
```

2 МЕТОД РЕШЕНИЯ

Класс cl_app:

- функционал:
    - метод get_handler — возвращает обработчик соотвествующего класса;
    - метод get_signal — возвращает сигнал соотвествующего класса;
    - метод build — строит дерево иерархии объектов;
    - метод exec — запуск программы.

Класс Base:

- свойства/поля:
    - поле номер класса:
        - наименование — number;
        - тип — int;
        - модификатор доступа — public;
- функционал:
    - метод addConnection — добавляет подключение к объекту;
    - теже методы что и в КВ_3
    - метод deleteConnection — удаляет подключение у объекта;
    - метод emitSignal — посылает сигнал объекту;
    - метод get_Path — возвращает абсолютный путь объекта.

Класс cl_1:

- функционал:
    - метод signal_f — метод сигнал;
    - метод handler_f — метод обработчика.

Класс cl_2:

- функционал:

    - метод signal_f — метод сигнал;
    - метод handler_f — метод обработчика.

Класс cl_3:

- функционал:
    - метод signal_f — метод сигнал;
    - метод handler_f — метод обработчика.

Класс cl_4:

- функционал:
    - метод signal_f — метод сигнал;
    - метод handler_f — метод обработчика.

Класс cl_5:

- функционал:
    - метод signal_f — метод сигнал;
    - метод handler_f — метод обработчика.

Класс cl_6:

- функционал:
    - метод signal_f — метод сигнал;
    - метод handler_f — метод обработчика.

Таблица 1 – Иерархия наследования классов

№	Имя класса	Классы-наследники	Модификатор доступа при наследовании	Описание	Номер
1	cl_app				
2	Base				
		cl_app	public		1
		cl_1	public		3
		cl_2	public		4
		cl_3	public		5

Продолжение Таблицы 1

		cl_4	public		6
		cl_5	public		7
		cl_6	public		8
3	cl_1				
4	cl_2				
5	cl_3				
6	cl_4				
7	cl_5				
8	cl_6				

3 ОПИСАНИЕ АЛГОРИТМОВ

Согласно этапам разработки, после определения необходимого инструментария в разделе «Метод», составляются подробные описания алгоритмов для методов классов и функций.

3.1 Алгоритм метода get_handler класса cl_app

Функционал: возвращает обработчик соотвествующего класса.

Параметры: int num-номер класса.

Возвращаемое значение: TYPE_HANDLER-обработчик соответсвющего класса.

Алгоритм метода представлен в таблице 2.

Таблица 2 – Алгоритм метода get_handler класса cl_app

№	Предикат	Действия	№ перехода
1	num равно 1	вернуть указатель на обработчик класса cl_1	∅
			2
2	num равно 2	вернуть указатель на обработчик класса cl_2	∅
			3
3	num равно 3	вернуть указатель на обработчик класса cl_3	∅
			4
4	num равно 4	вернуть указатель на обработчик класса cl_4	∅
			5
5	num равно 5	вернуть указатель на обработчик класса cl_5	∅
			6
6	num равно 6	вернуть указатель на обработчик класса cl_6	∅
			7
7		вернуть нулевой указатель	∅

3.2 Алгоритм метода get_signal класса cl_app

Функционал: возвращает сигнал соотвествующего класса.

Параметры: int num.

Возвращаемое значение: TYPE_SIGNAL.

Алгоритм метода представлен в таблице 3.

Таблица 3 – Алгоритм метода get_signal класса cl_app

№	Предикат	Действия	№ перехода
1	num равно 1	вернуть указатель на сигнал класса cl_1	∅
			2
2	num равно 2	вернуть указатель на сигнал класса cl_2	∅
			3
3	num равно 3	вернуть указатель на сигнал класса cl_3	∅
			4
4	num равно 4	вернуть указатель на сигнал класса cl_4	∅
			5
5	num равно 5	вернуть указатель на сигнал класса cl_5	∅
			6
6	num равно 6	вернуть указатель на сигнал класса cl_6	∅
			7
7		вернуть нулевой указатель	∅

3.3 Алгоритм метода build класса cl_app

Функционал: строит дерево иерархии объектов.

Параметры: нет.

Возвращаемое значение: void.

Алгоритм метода представлен в таблице 4.

Таблица 4 – Алгоритм метода build класса cl_app

№	Предикат	Действия	№ перехода
1		объявляются строковые переменные root_name, parent_coords,child_name	2
2		инициализируем указатели parent и child на объекты класса Base указателем на текущий объект	3
3		объявить целое число clNumber	4
4		Ввод root_name	5
5		Вызов метода setName с параметром root_name	6
6		Ввод parent_coords	7
7	parent_coords равно endtree	остановка цикла	21
			8
8		ввод child_name и clNumber	9
9		присвоить parent результат работы findByCoords с параметром parent_coords	10
10	parent не равен нулевому указателю и результат работы метода getChildByName объекта с адресо parent с аргументом child_name равен нулевому указателю		11
			16
11	clNumber равен 2	присвоить переменной child значение указателя на созданный объект cl_2 с аргументами parent и child_name	6
			12

Продолжение Таблицы 4

12	clNumber равен 3	присвоить переменной child значение указателя на созданный объект cl_3 с аргументами parent и child_name	6
			13
13	clNumber равен 4	присвоить переменной child значение указателя на созданный объект cl_4 с аргументами parent и child_name	6
			14
14	clNumber равен 5	присвоить переменной child значение указателя на созданный объект cl_5 с аргументами parent и child_name	6
			15
15	clNumber равен 6	присвоить переменной child значение указателя на созданный объект cl_6 с аргументами parent и child_name	6
			6
16	parent равно нулевому указателю	Вывод "Object tree "	17
			20
17		вызов метода print	18
18		Вывод "The Head Object ",parent_coords,"is not found"	19
19		выход с кодом 1	∅
20		вывод parent_coords и "Dubbing the names of subordinate objects"	6
21		объявить указатель sender на объект класса Base	22
22		объявить указатель receiver на объект класса Base	23
23		объявить строковые переменные sender_name и handler_name	24

Продолжение Таблицы 4

24		ввод sender_name	25
25	sender_name равно "end_of_connections"	выход из цикла	∅
			26
26		ввод receiver_name	27
27		присвоить sender результат работы метод findByCoords с аргументом sender_name	28
28		присвоить reciever результат работы метод findByCoords с аргументом receiver_name	29
29		вызов метода addConnection с аргументами get_signal с номером класса sender,receiver и get_handler с номером класса receiver	24

3.4 Алгоритм метода exec класса cl_app

Функционал: запуск программы.

Параметры: нет.

Возвращаемое значение: int.

Алгоритм метода представлен в таблице 5.

Таблица 5 – Алгоритм метода exec класса cl_app

№	Предикат	Действия	№ перехода
1		вывод "Object tree "	2
2		вызов метода print	3
3		объявить строковые переменные coomand, message, coords	4
4		ввод coomand	5
5	command не равно "END"	ввод coords	6
			19

Продолжение Таблицы 5

6		инициализировать указатель obj на объект Base результатом работы метода findByCoords с аргументом coords	7
7		вызов функции getline с аргументами cin и message	8
8	obj равно нулевому указателю	Вывод сообщения о том, что объект по заданным координатам не был найдет	9
			10
9		ввод command	5
10	coomand равно "EMIT"	вызов метода emitSignal у объекта с адресом obj с аргументами get_signal с номером класса и message	18
			11
11	command равно "SET_CONDITION"	вызов метода setState с аргументом stoi и message	18
			12
12		присвоить message результат работы метода substr переменной message с аргументом 1	13
13		инциализировать указатель c на объект класса Base результатом работы метода findByCoords с аргументом message	14
14	c равно нулевому указателю	вывод "Handler object ", coords,"not found"	15
			16
15		ввод coomand	5
16	command равно "SET_CONNECT"	вызов метода deleteConnection объекта с адресом obj с аргументами get_signal с номером класса obj, c,get_handler с номером класса obj	18
			17

Продолжение Таблицы 5

17	coomand равно "DELETE_CONNECT"	вызов метода deleteConnection объекта с адресом obj с аргументами get_signal с номером класса obj,c, get_handler с номером класса obj	18
			18
18		ввод command	5
19		вернуть 0	∅

3.5 Алгоритм метода addConnection класса Base

Функционал: добавляет подключение к объекту.

Параметры: TYPE_SIGNAL signal- сигнал,указатель base на объекта класса Base,TYPE_HANDLER handler - обработчик.

Возвращаемое значение: void.

Алгоритм метода представлен в таблице 6.

Таблица 6 – Алгоритм метода addConnection класса Base

№	Предикат	Действия	№ перехода
1		объявить указатель connection на структуру o_sh	2
2	перебираем элементы вектора connects. i-индекс элемента		3
			4
3	поле p_signal i-го элемента вектора connects равно signal и p_base i-го элемента вектора connects равно base и поле p_handler i-го элемента вектора connects равно handler		∅

Продолжение Таблицы 6

			2
4		присвоить connection значение адреса созданного объекта o_sh	5
5		присвоить полю p_signal объекта с адресом connection значение signal	6
6		присвоить полю p_base объекта с адресом connection значение base	7
7		присвоить полю p_handler объекта с адресом connection значение handler	8
8		вызов метода push_back вектора connects с аргументом connection	∅

3.6 Алгоритм метода deleteConnection класса Base

Функционал: удаляет подключение у объекта.

Параметры: TYPE_SIGNAL,base,TYPE_HANDLER.

Возвращаемое значение: void.

Алгоритм метода представлен в таблице 7.

Таблица 7 – Алгоритм метода deleteConnection класса Base

№	Предикат	Действия	№ перехода
1	перебираем элементы вектора connects. i-индекс элемента		2
			∅

Продолжение Таблицы 7

2	поле p_signal i-го элемента вектора connects равно base и поле p_handler i-го элемента вектора connects равно handler	инициализировать указатель found на объект структуры o_sh i-ым элементом вектора connects	3
			1
3		вызов метода erase вектора connects с аргументом connects.begin() плюс i	4
4		удалить указатель found	∅

3.7 Алгоритм метода emitSignal класса Base

Функционал: посылает сигнал объекту.

Параметры: TYPE_SIGNAL signal, string& command.

Возвращаемое значение: void.

Алгоритм метода представлен в таблице 8.

Таблица 8 – Алгоритм метода emitSignal класса Base

№	Предикат	Действия	№ перехода
1	_state не равно 0	Вызов метода сигнала с аргументом command	2
			∅
2	перебираем элементы вектора connects. i-индекс элемента		3
			∅
3	поле p_signal i-го элемента вектора connects равно signal	инциализировать объект handler типа TYPE_HANDLER полем p_handler i-го элемента вектора connects	4
			2

Продолжение Таблицы 8

4		инициализировать указатель base на объект класса Base полем p_base i-го элемента вектора connects	5
5	поле _state объекта с адресом base не равно 0	Вызов метода обработчика с аргументом command	2
			2

3.8 Алгоритм метода get_Path класса Base

Функционал: возвращает абсолютный путь объекта.

Параметры: нет.

Возвращаемое значение: string.

Алгоритм метода представлен в таблице 9.

Таблица 9 – Алгоритм метода get_Path класса Base

№	Предикат	Действия	№ перехода
1	parent не равно нулевому указателю		2
			3
2	поле parent объекта parent не равно нулевому указателю	Вернуть результат работы метода get_path объекта parent плюс "/" плюс _name	∅
		вернуть результат работы метода get_path объекта parent плюс _name	∅
3		Вернуть "/"	∅

3.9 Алгоритм метода signal_f класса cl_1

Функционал: метод сигнал.

Параметры: string& message.

Возвращаемое значение: void.

Алгоритм метода представлен в таблице 10.

Таблица 10 – Алгоритм метода signal_f класса cl_1

№	Предикат	Действия	№ перехода
1		Вывод "Signal from ", результат работы метода get_path	2
2		Прибавить к message значение " (class: 1)"	∅

3.10 Алгоритм метода handler_f класса cl_1

Функционал: метод обработчика.

Параметры: string message.

Возвращаемое значение: void.

Алгоритм метода представлен в таблице 11.

Таблица 11 – Алгоритм метода handler_f класса cl_1

№	Предикат	Действия	№ перехода
1		Вывод "Signal to ", результат работы метода get_path,"Text: ",message	∅

3.11 Алгоритм метода signal_f класса cl_2

Функционал: метод сигнал.

Параметры: string& message.

Возвращаемое значение: void.

Алгоритм метода представлен в таблице 12.

Таблица 12 – Алгоритм метода signal_f класса cl_2

№	Предикат	Действия	№ перехода
1		Вывод "Signal from ", результат работы метода get_path	2
2		Прибавить к message значение " (class: 2)"	∅

3.12 Алгоритм метода handler_f класса cl_2

Функционал: метод обработчика.

Параметры: string message.

Возвращаемое значение: void.

Алгоритм метода представлен в таблице 13.

Таблица 13 – Алгоритм метода handler_f класса cl_2

№	Предикат	Действия	№ перехода
1		Вывод "Signal to ", результат работы метода get_path,"Text: ",message	∅

3.13 Алгоритм метода signal_f класса cl_3

Функционал: метод сигнал.

Параметры: messgae&.

Возвращаемое значение: void.

Алгоритм метода представлен в таблице 14.

Таблица 14 – Алгоритм метода signal_f класса cl_3

№	Предикат	Действия	№ перехода
1		Вывод "Signal from ", результат работы метода get_path	2
2		Прибавить к message значение " (class: 3)"	∅

3.14 Алгоритм метода handler_f класса cl_3

Функционал: метод обработчика.

Параметры: нет.

Возвращаемое значение: void.

Алгоритм метода представлен в таблице 15.

Таблица 15 – Алгоритм метода handler_f класса cl_3

№	Предикат	Действия	№ перехода
1		Вывод "Signal to ", результат работы метода get_path,"Text: ",message	∅

3.15 Алгоритм метода signal_f класса cl_4

Функционал: метод сигнал.

Параметры: string& message.

Возвращаемое значение: void.

Алгоритм метода представлен в таблице 16.

Таблица 16 – Алгоритм метода signal_f класса cl_4

№	Предикат	Действия	№ перехода
1		Вывод "Signal from ", результат работы метода get_path	2
2		Прибавить к message значение " (class: 4)"	∅

3.16 Алгоритм метода handler_f класса cl_4

Функционал: метод обработчика.

Параметры: string message.

Возвращаемое значение: void.

Алгоритм метода представлен в таблице 17.

Таблица 17 – Алгоритм метода handler_f класса cl_4

№	Предикат	Действия	№ перехода
1		Вывод "Signal to ", результат работы метода get_path,"Text: ",message	∅

3.17 Алгоритм метода signal_f класса cl_5

Функционал: метод сигнал.

Параметры: string& message.

Возвращаемое значение: void.

Алгоритм метода представлен в таблице 18.

Таблица 18 – Алгоритм метода signal_f класса cl_5

№	Предикат	Действия	№ перехода
1		Вывод "Signal from ", результат работы метода get_path	2
2		Прибавить к message значение " (class: 5)"	∅

3.18 Алгоритм метода handler_f класса cl_5

Функционал: метод обработчика.

Параметры: string message.

Возвращаемое значение: void.

Алгоритм метода представлен в таблице 19.

Таблица 19 – Алгоритм метода handler_f класса cl_5

№	Предикат	Действия	№ перехода
1		Вывод "Signal to ", результат работы метода get_path,"Text: ",message	∅

3.19 Алгоритм метода signal_f класса cl_6

Функционал: метод сигнал.

Параметры: string&.

Возвращаемое значение: void.

Алгоритм метода представлен в таблице 20.

Таблица 20 – Алгоритм метода signal_f класса cl_6

№	Предикат	Действия	№ перехода
1		Вывод "Signal from ", результат работы метода get_path	2

Продолжение Таблицы 20

2		Прибавить к message значение " (class: 6)"	∅

3.20 Алгоритм метода handler_f класса cl_6

Функционал: метод обработчика.

Параметры: string message.

Возвращаемое значение: void.

Алгоритм метода представлен в таблице 21.

Таблица 21 – Алгоритм метода handler_f класса cl_6

№	Предикат	Действия	№ перехода
1		Вывод "Signal to ", результат работы метода get_path,"Text: ",message	∅

4 БЛОК-СХЕМЫ АЛГОРИТМОВ

Представим описание алгоритмов в графическом виде на рисунках 1-25.

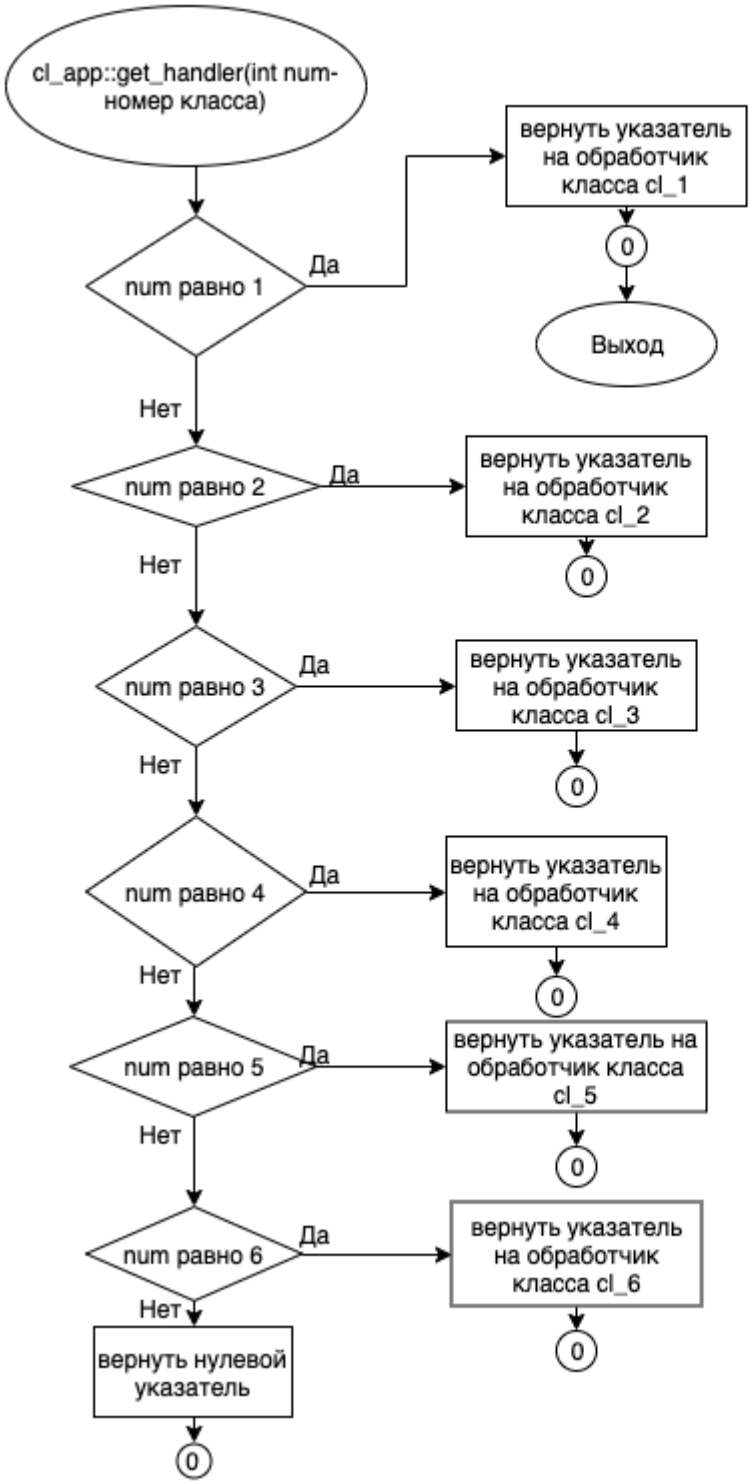

Рисунок 1 – Блок-схема алгоритма

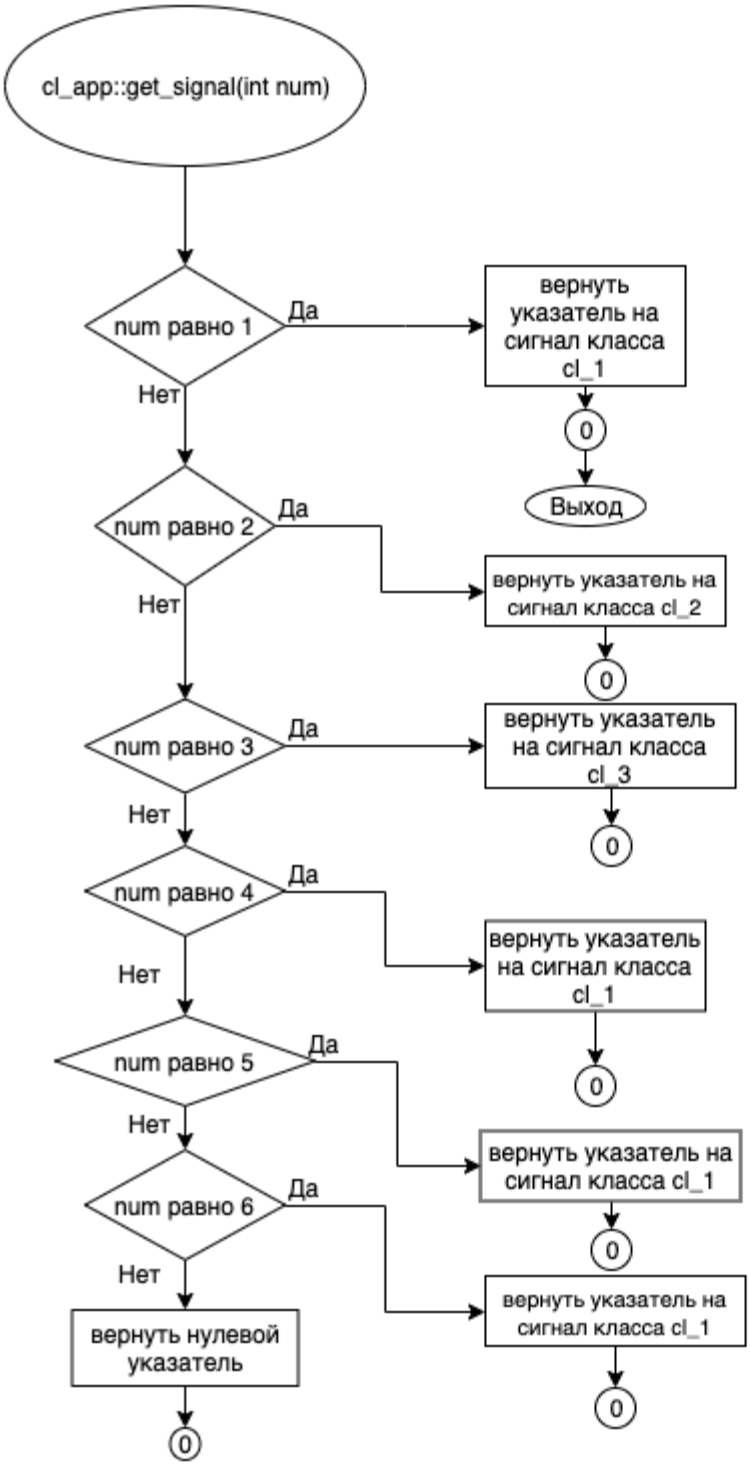

Рисунок 2 – Блок-схема алгоритма

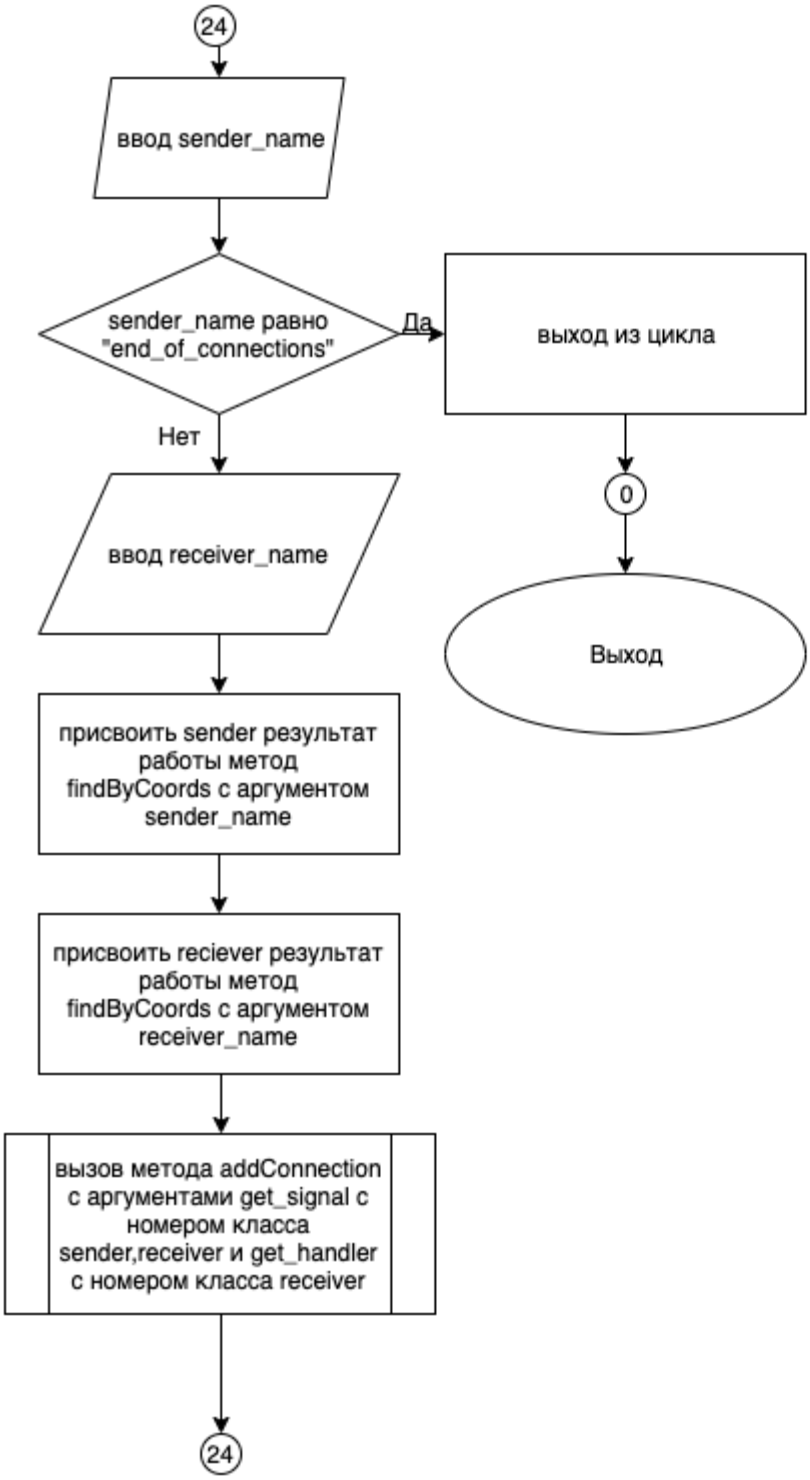

Рисунок 3 – Блок-схема алгоритма

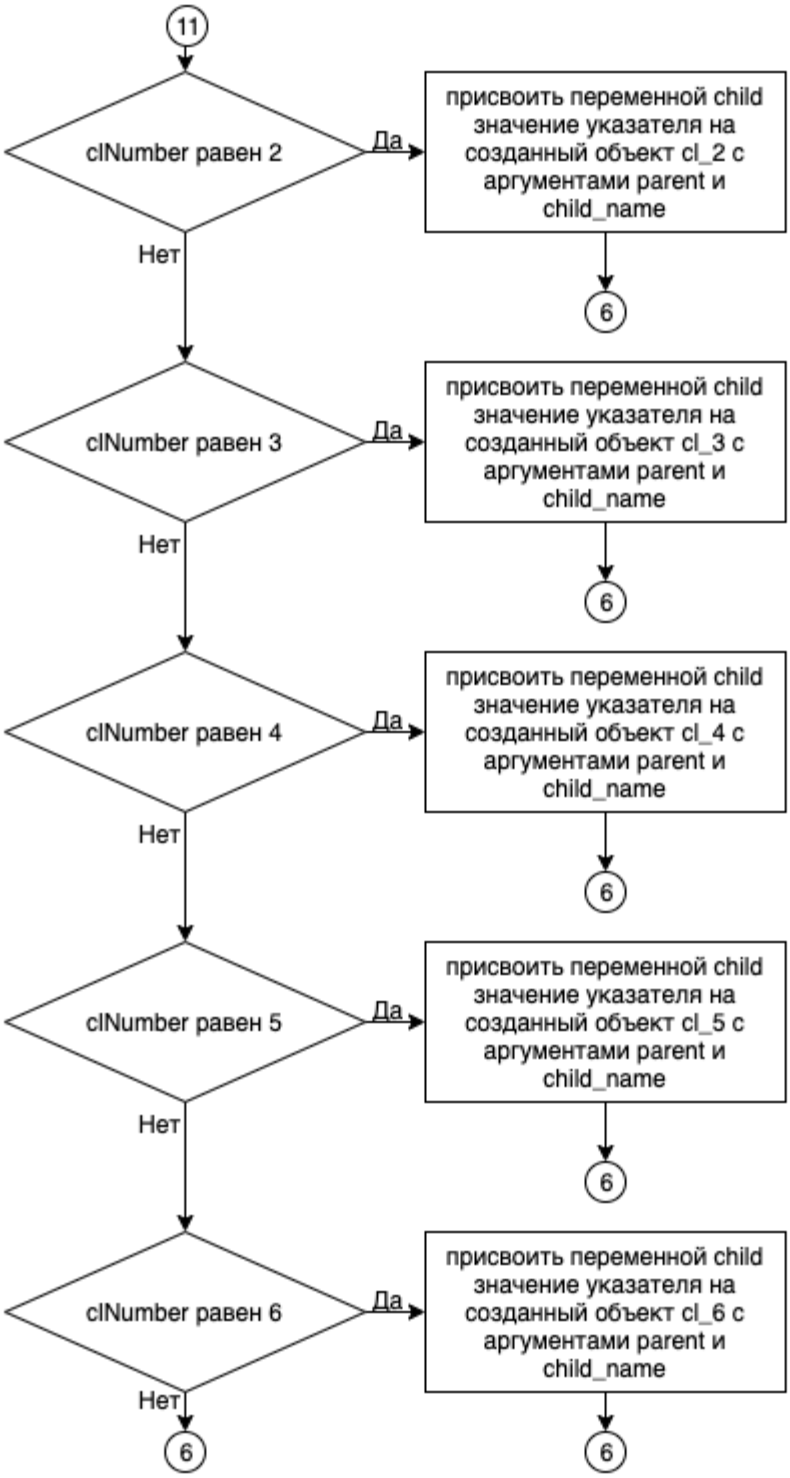

Рисунок 4 – Блок-схема алгоритма

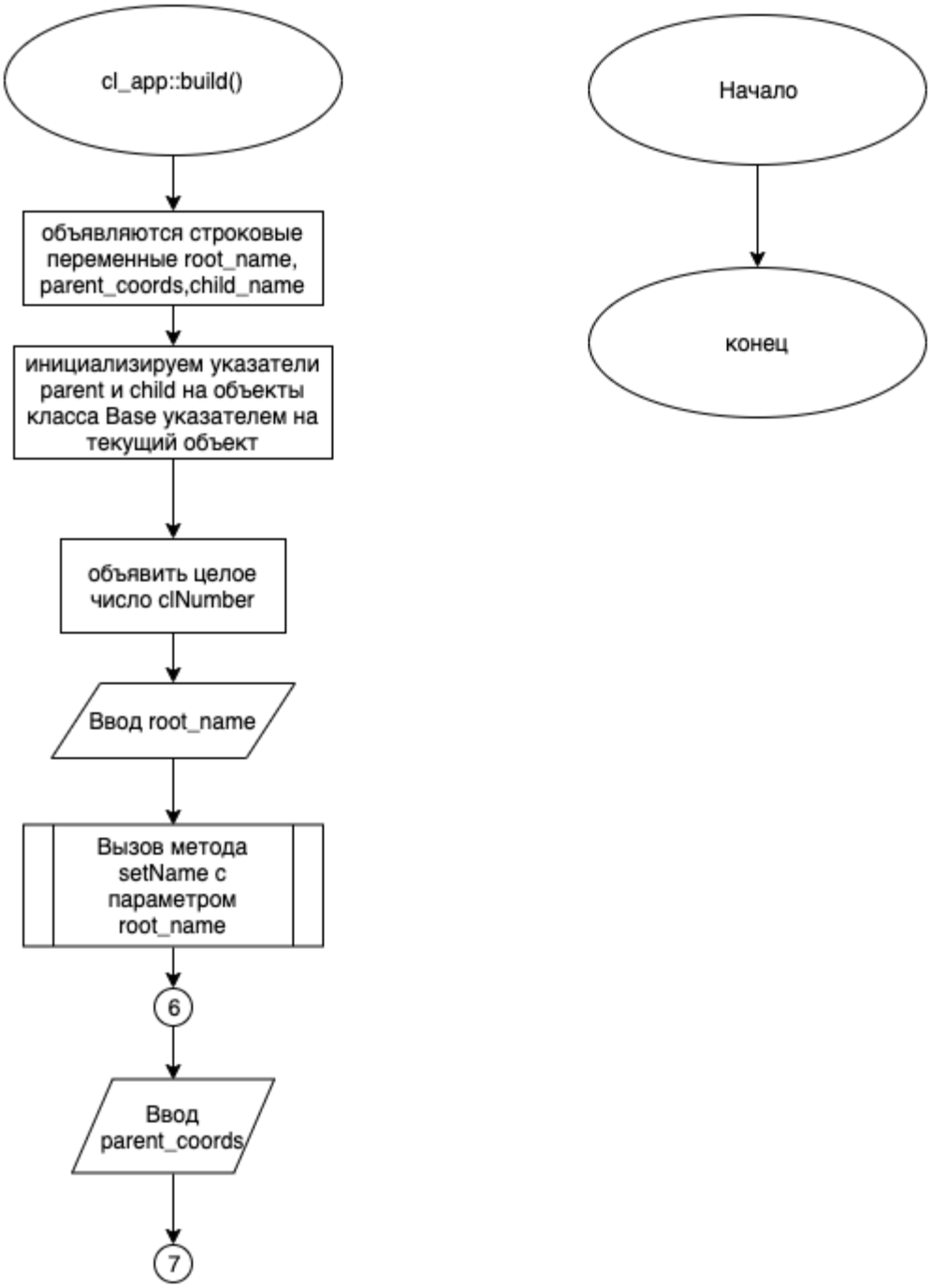

Рисунок 5 – Блок-схема алгоритма

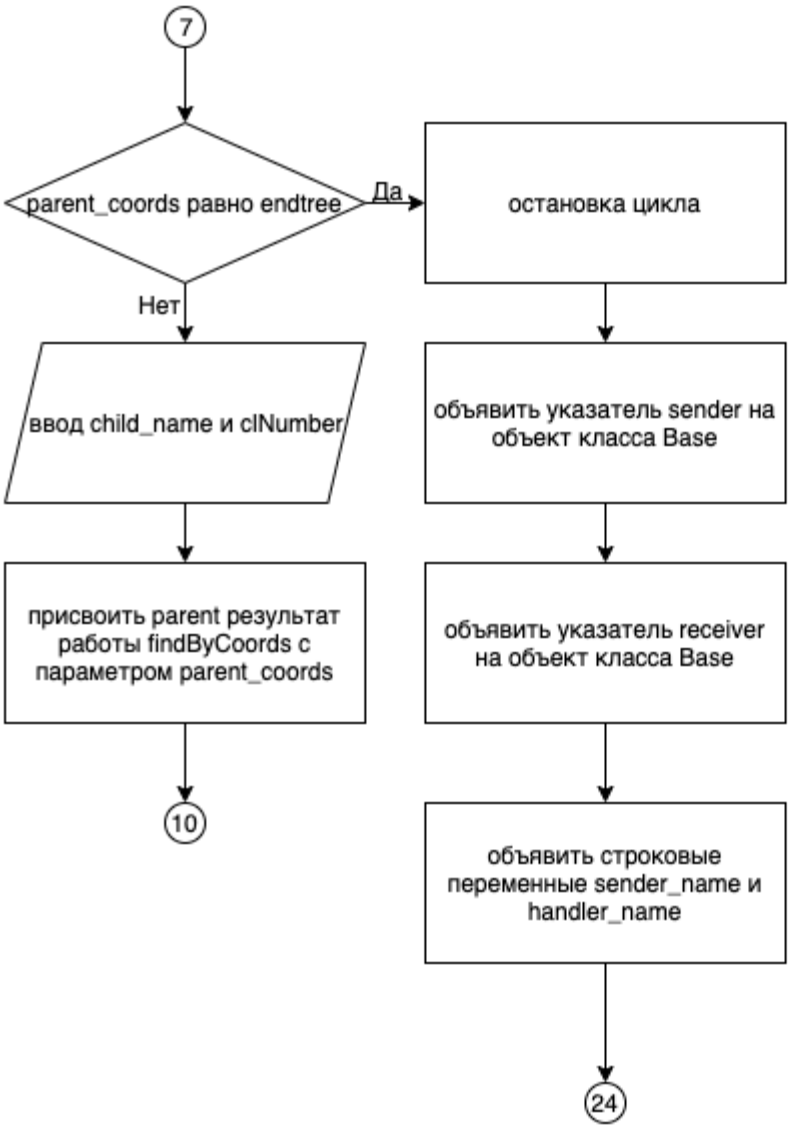

Рисунок 6 – Блок-схема алгоритма

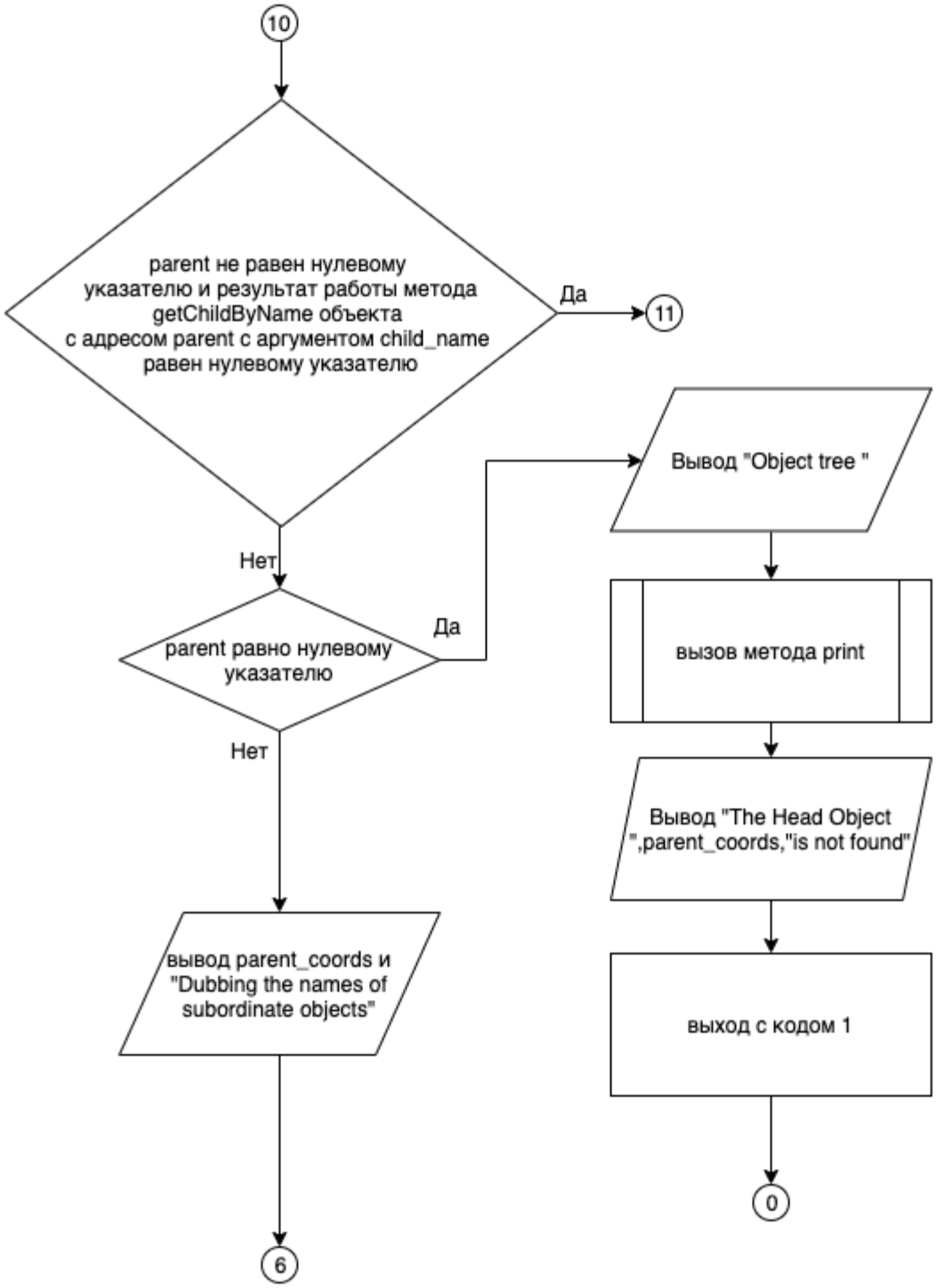

Рисунок 7 – Блок-схема алгоритма

Рисунок 8 – Блок-схема алгоритма

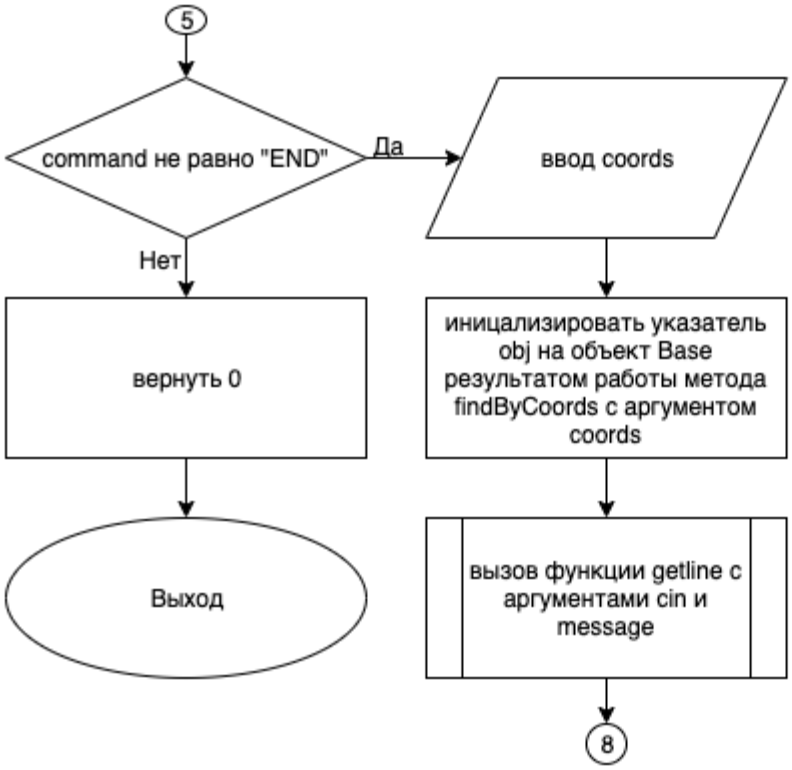

Рисунок 9 – Блок-схема алгоритма

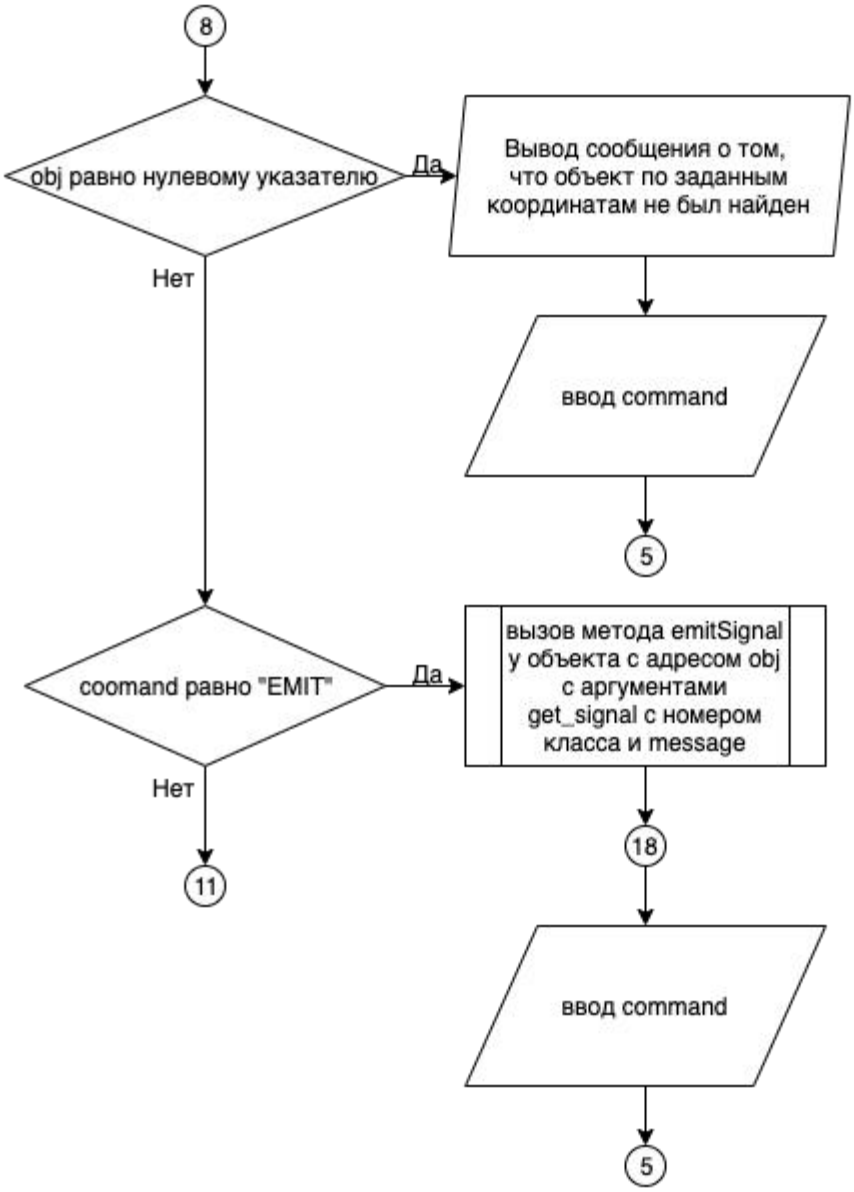

Рисунок 10 – Блок-схема алгоритма

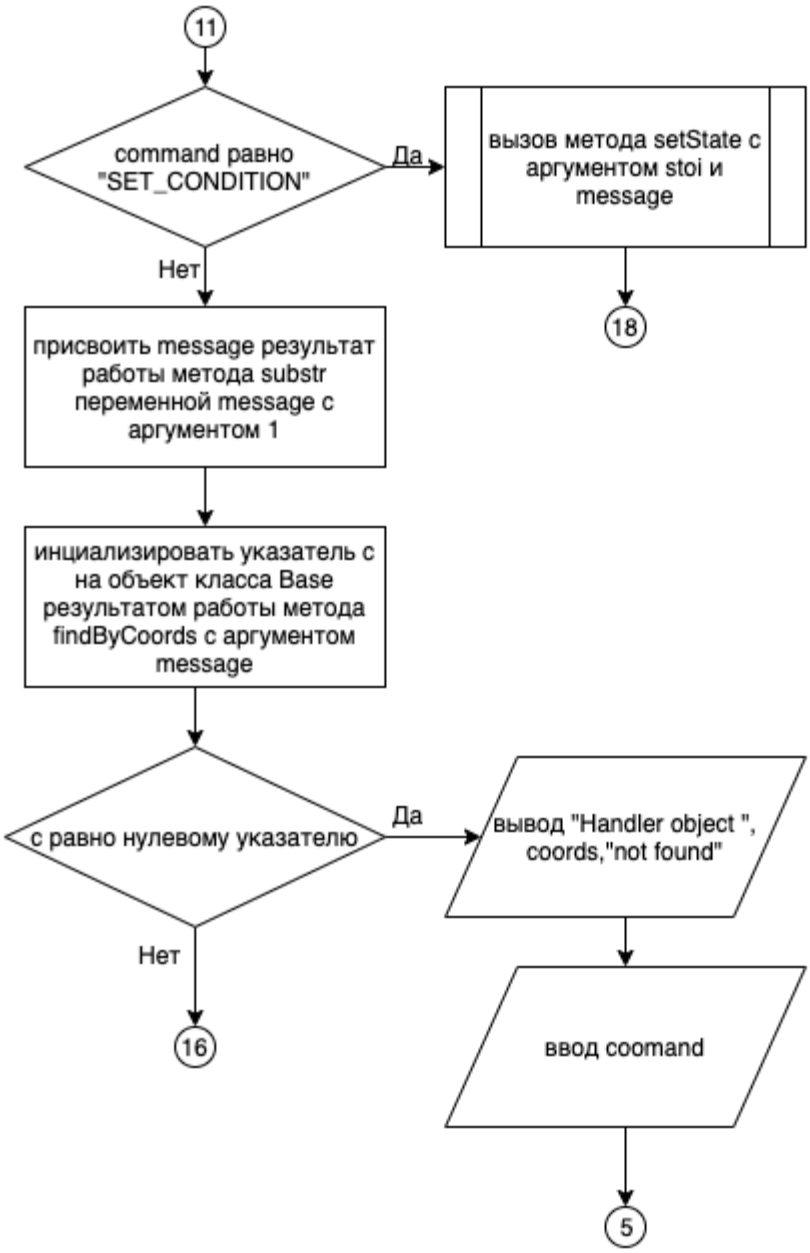

Рисунок 11 – Блок-схема алгоритма

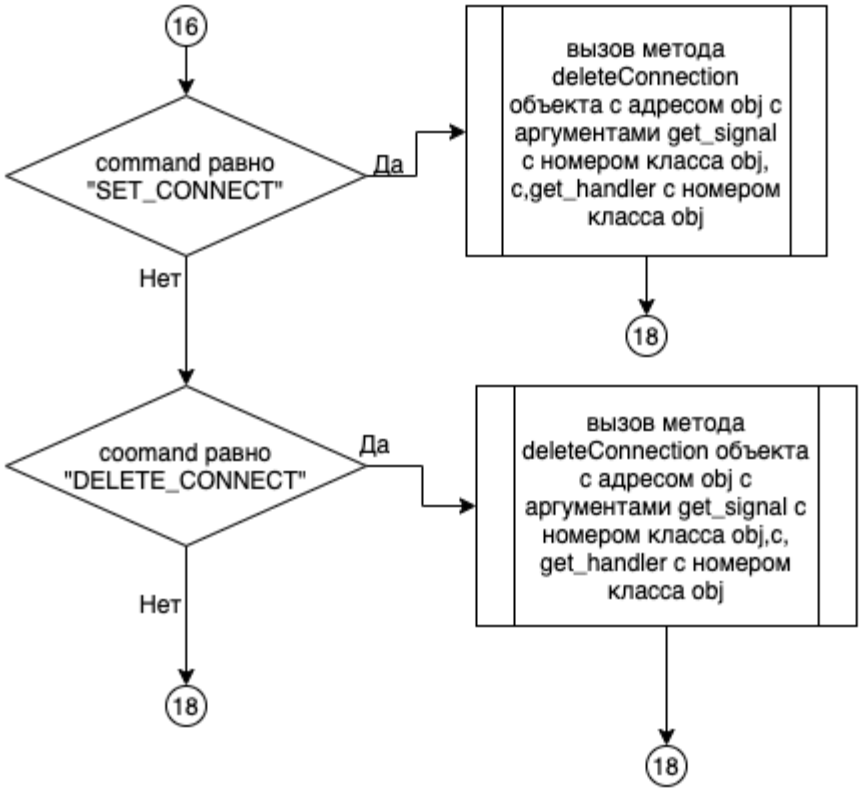

Рисунок 12 – Блок-схема алгоритма

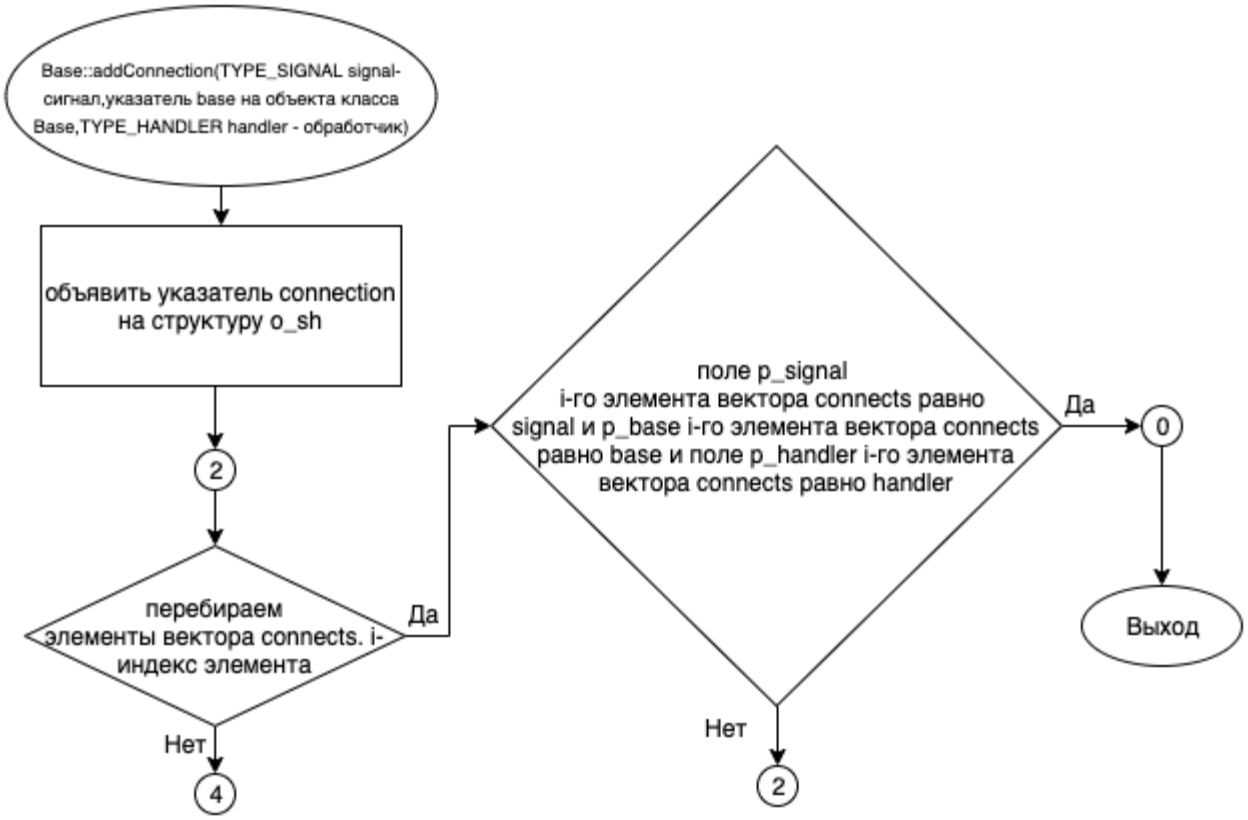

Рисунок 13 – Блок-схема алгоритма

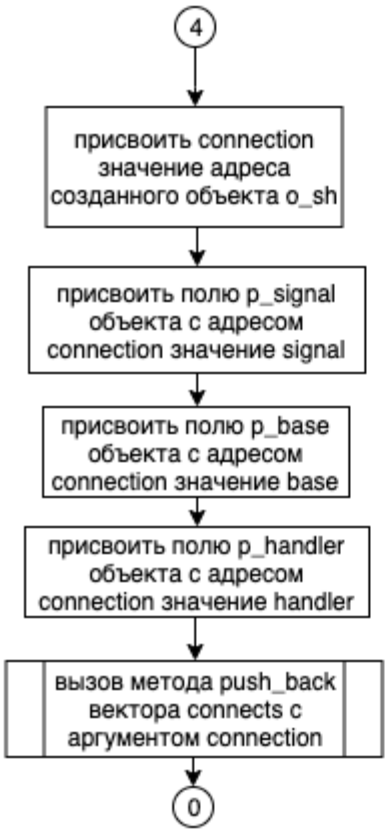

Рисунок 14 – Блок-схема алгоритма

Рисунок 15 – Блок-схема алгоритма

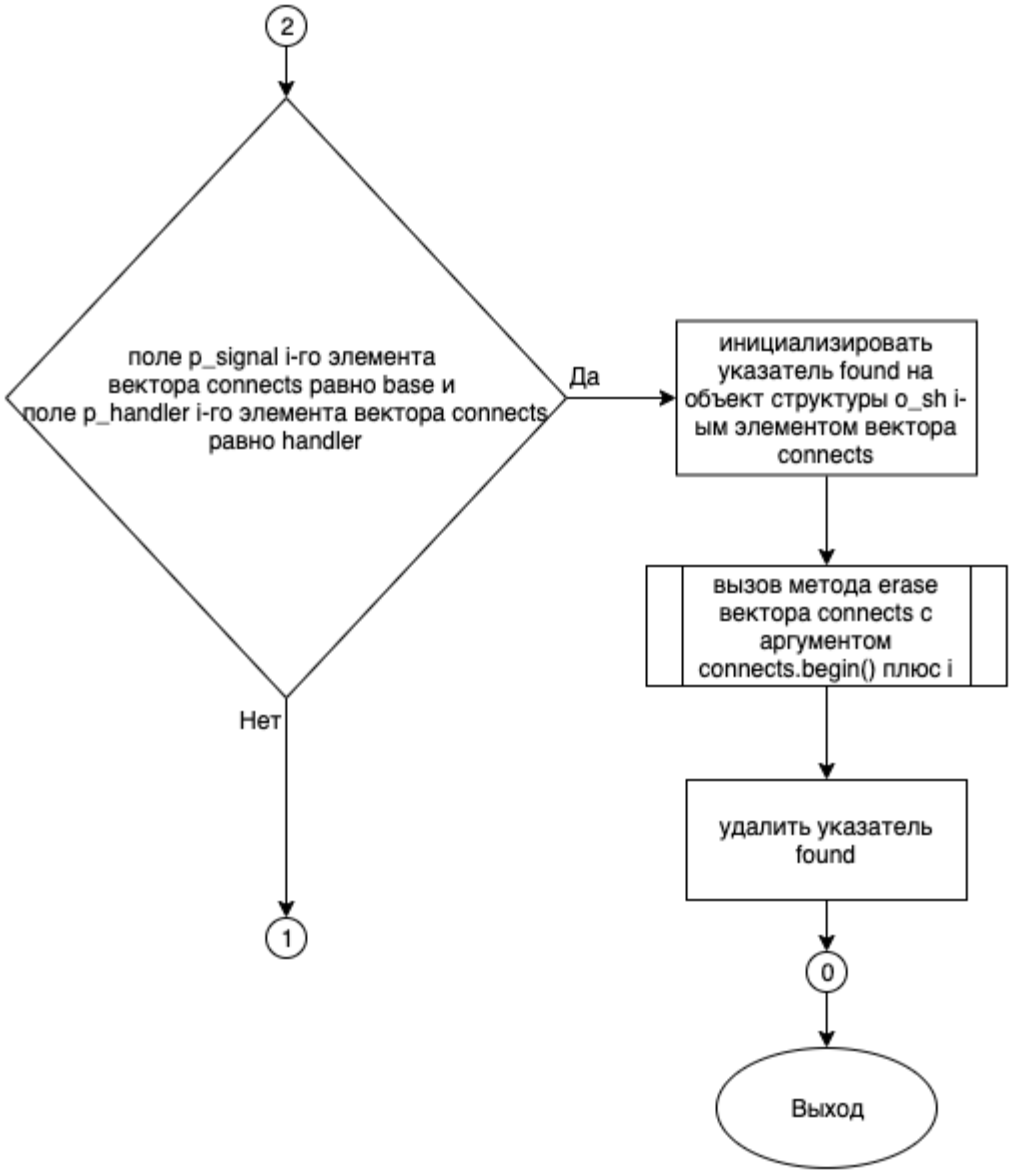

Рисунок 16 – Блок-схема алгоритма

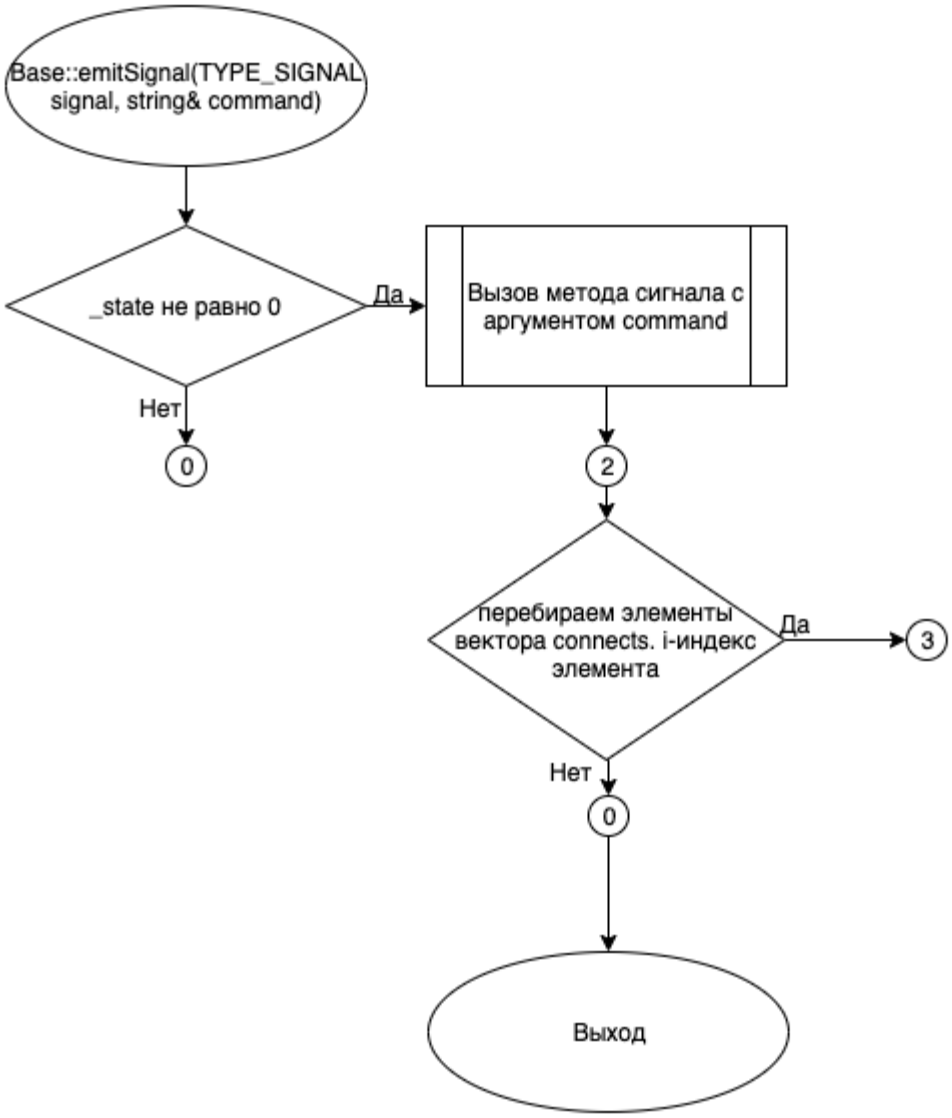

Рисунок 17 – Блок-схема алгоритма

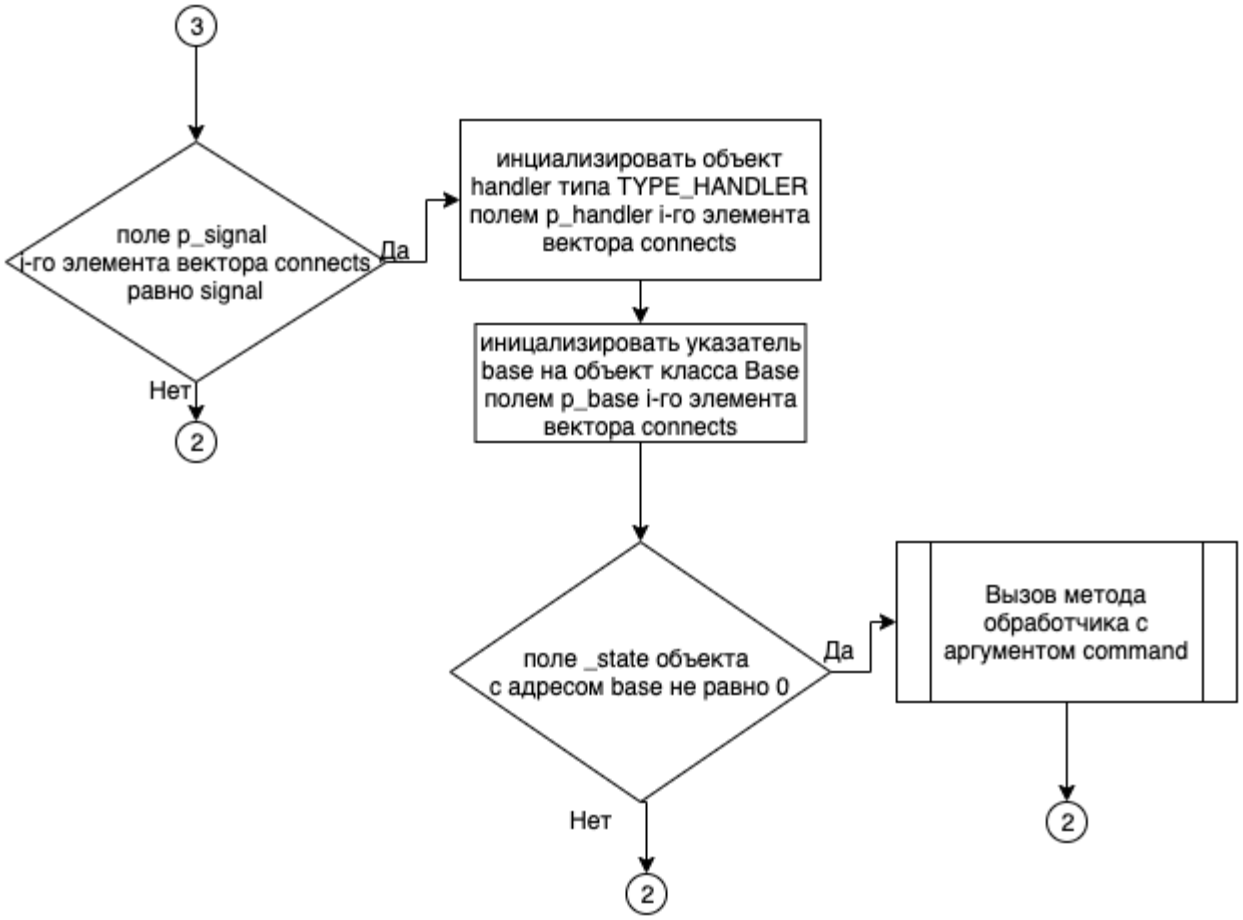

Рисунок 18 – Блок-схема алгоритма

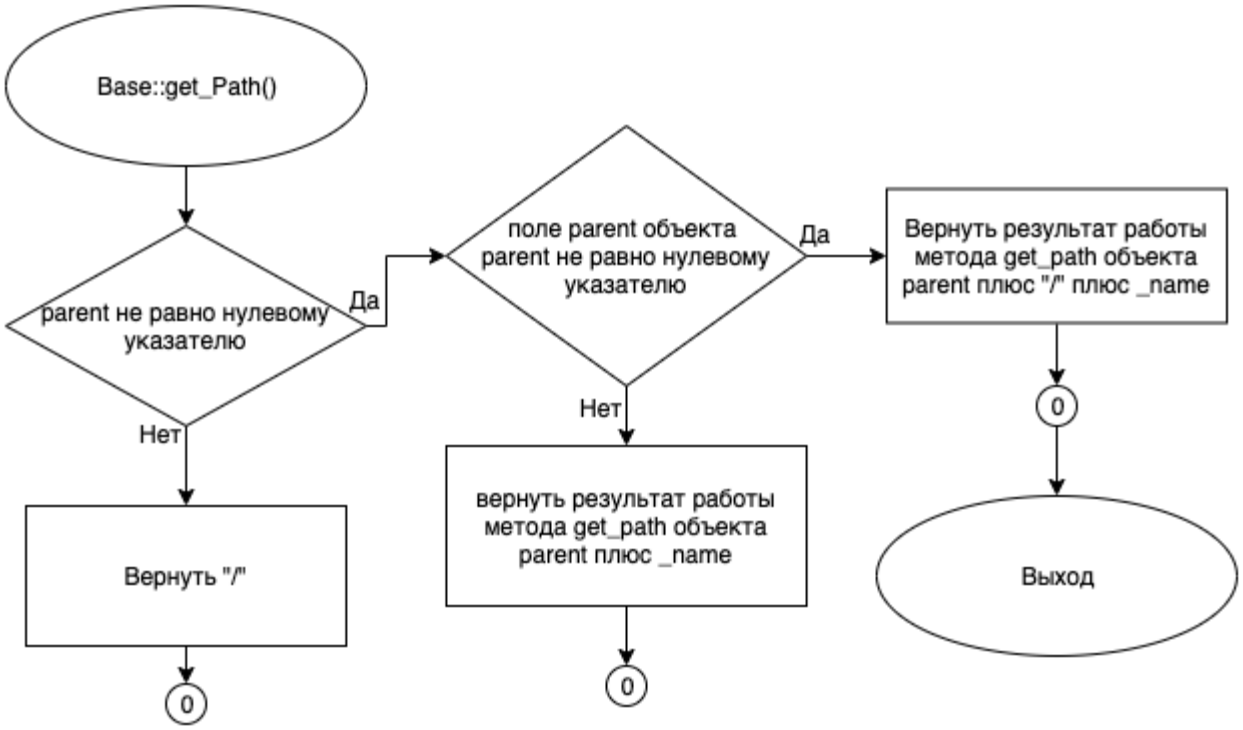

Рисунок 19 – Блок-схема алгоритма

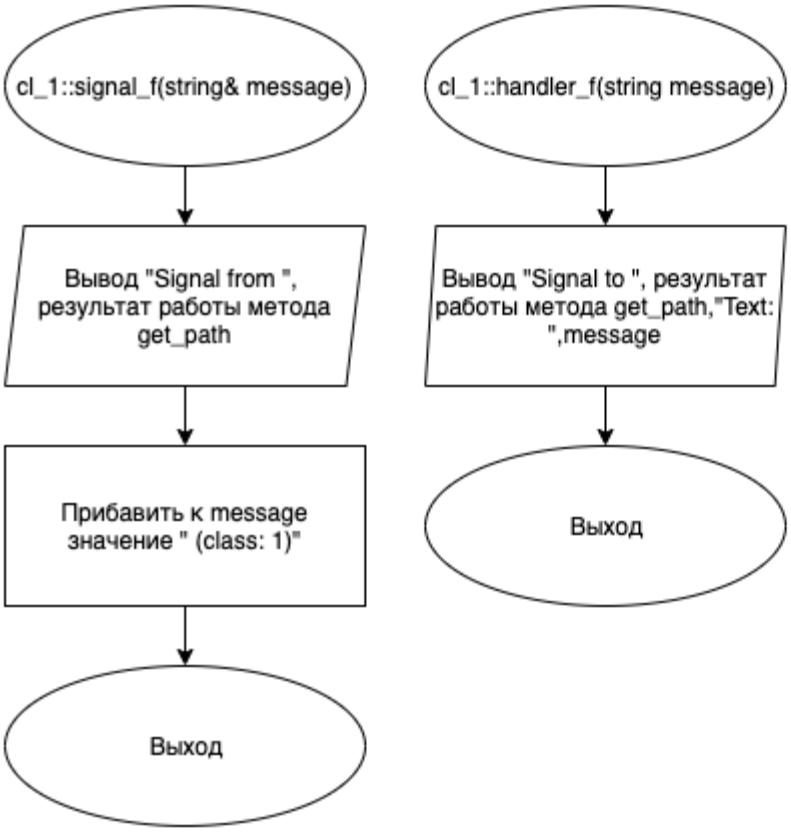

Рисунок 20 – Блок-схема алгоритма

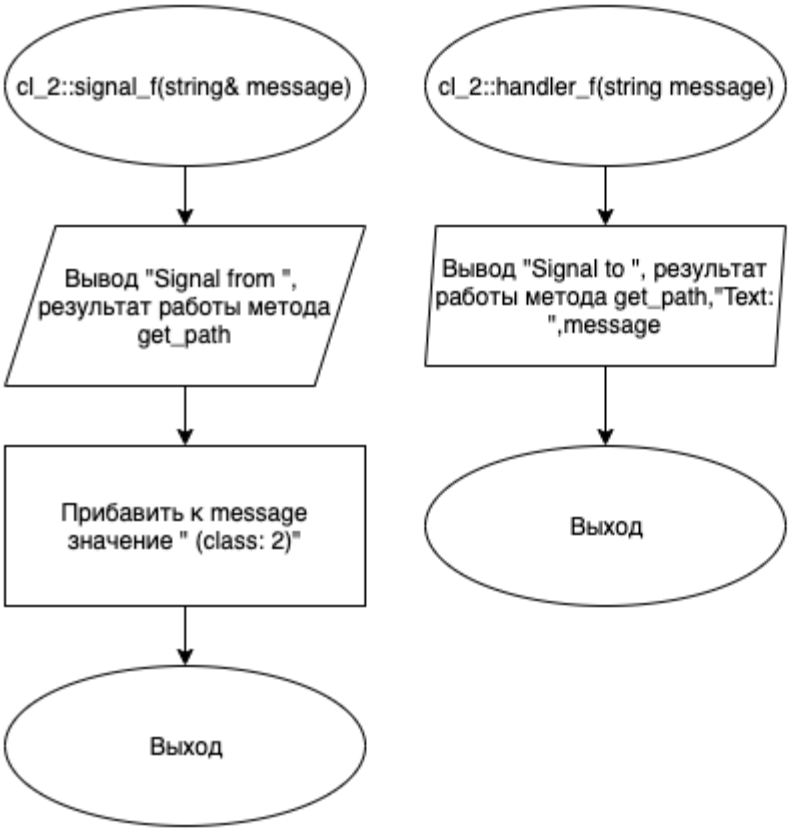

Рисунок 21 – Блок-схема алгоритма

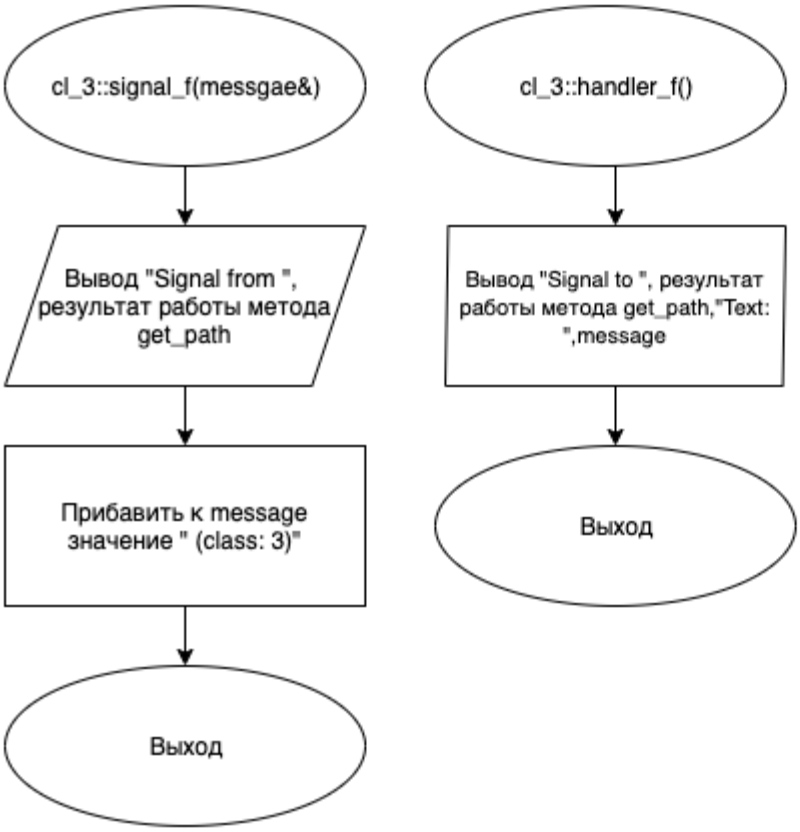

Рисунок 22 – Блок-схема алгоритма

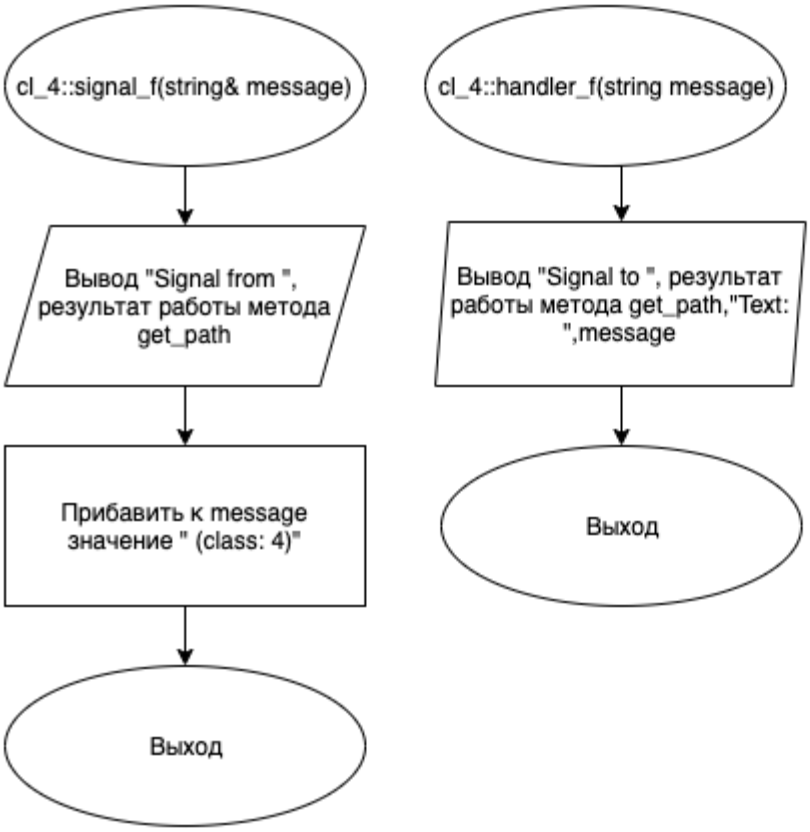

Рисунок 23 – Блок-схема алгоритма

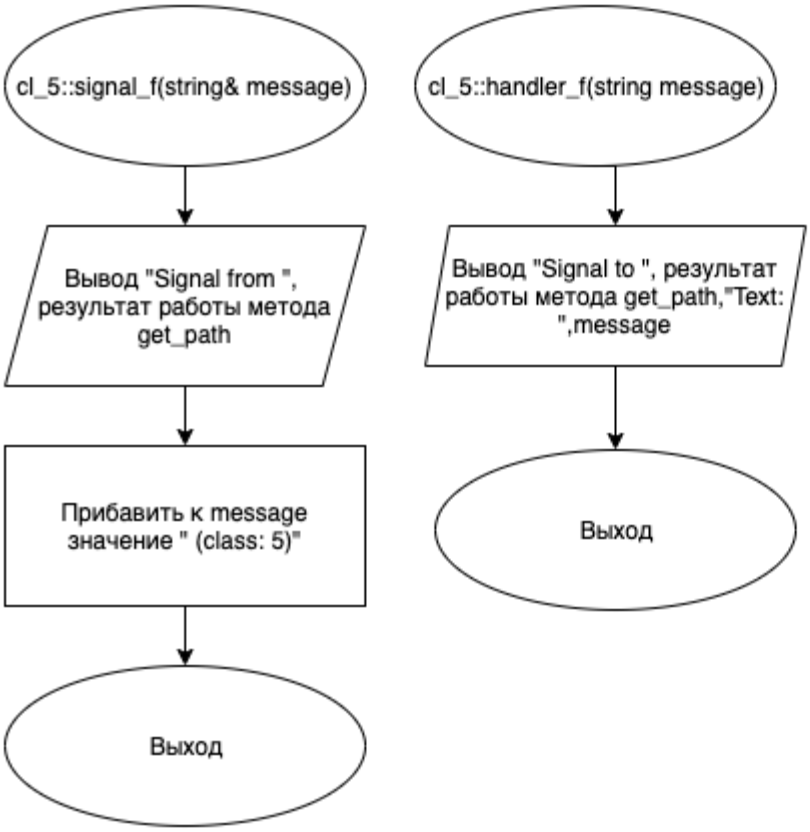

Рисунок 24 – Блок-схема алгоритма

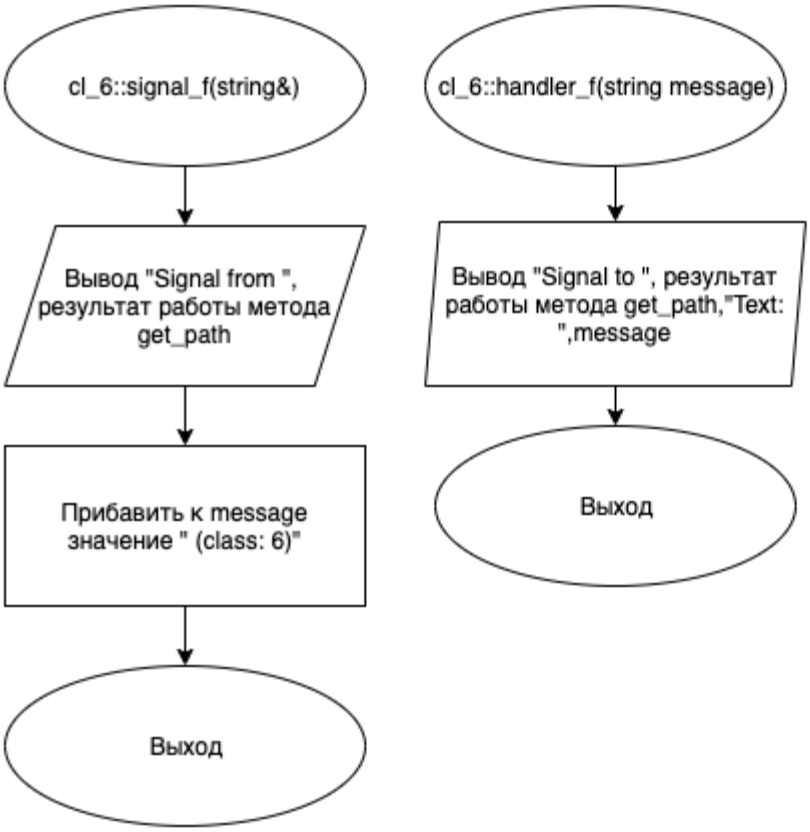

Рисунок 25 – Блок-схема алгоритма

ЗАКЛЮЧЕНИЕ

В ходе работы научились определять указатель на объект по его координате. АРМ Аврора делает разработку более комфортной. Облегчается работа с тестированием благодарая сохранению текстов и во время разработки ввода данных. Алгоритм,метода решения. блок-схема способствую более глубокому пониманию объектно-ориентированного программирования. Генерирование отчёта также упрощает работу.[5] Разработка данной программы позволила мне закрепить материал, полученный в процессе изучения курса, освоить принципы объектно-ориентированного программирования и убедиться в его значимости.[6]

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19 Единая система программной документации.

2. Методическое пособие студента для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [Электронный ресурс] – URL: https://mirea.aco-avrora.ru/student/files/methodichescoe_posobie_dlya_laboratornyh_rabot_3.pdf (дата обращения 05.05.2021).

3. Приложение к методическому пособию студента по выполнению заданий в рамках курса «Объектно-ориентированное программирование» [Электронный ресурс]. URL: https://mirea.aco-avrora.ru/student/files/Prilozheniye_k_methodichke.pdf (дата обращения 05.05.2021).

4. Шилдт Г. C++: базовый курс. 3-е изд. Пер. с англ.. — М.: Вильямс, 2019. — 624 с.

5. Видео лекции по курсу «Объектно-ориентированное программирование» [Электронный ресурс]. АСО «Аврора».

6. Антик М.И. Дискретная математика [Электронный ресурс]: Учебное пособие /Антик М.И., Казанцева Л.В. — М.: МИРЭА — Российский технологический университет, 2018 — 1 электрон. опт. диск (CD-ROM).

ПРИЛОЖЕНИЕ 1. КОД ПРОГРАММЫ

Программная реализация алгоритмов для решения задачи представлена ниже.

Файл Base.cpp

Листинг 1 – Base.cpp

```
#include "Base.h"
#include <iostream>
#include <algorithm>
Base::Base(Base* _parent, std::string name): parent(_parent), _name(name){
   if(_parent != nullptr){
      _parent->children.push_back(this); // Когда создаем узел, добавляем к
родителю (если он есть) текущий узел
   }
}
Base::~Base(){
   for(int i = 0;i < children.size();i++){
      delete children[i]; // Удаляя родителя, удалем всех его детей
   }
}
bool Base::setName(std::string name){
   if(findOnTreeByName(name)  !=  nullptr){  //  Проверка,  уникально  ли  имя
объекта в дереве
         return false;
   }else{
      _name = name;
      return true;
   }
}
void Base::setState(int state){
   if(state != 0){
      if(parent  ==  nullptr  ||  parent->_state  !=  0){  // Если все объекты в
иерархии,  находящиеся  выше  готовы  -  значит  можно  выставлять  готовность
текущему
            _state = state;
      }
   }else{
      _state = 0;
      for(int i = 0;i < children.size();i++){
            children[i]->setState(state);   //   Рекурсивно   выключаем   все
объекты, находящиеся ниже в иерархии
         }
      }
}
std::string Base::getName(){
   return _name;
```

```
}
Base* Base::getParent(){
   return parent;
}
Base* Base::getChildByName(std::string name){ // Поиск ребенка по имени
   for(int i = 0;i < children.size();i++){
      if(children[i]->getName() == name){
         return children[i];
      }
   }
   return nullptr;
}
int _foundByNameCount = 0;
Base* _foundChild = nullptr;
Base* Base::findByName(std::string name){ // Поиск объекта по имени ниже в
иерархии
   if(_name == name){ // Если имя текущего объекта совпадает с найденным,
увеличиваем счетчик и сохраняем найденный объект
      _foundByNameCount++;
      _foundChild = this;
   }
   for(int i = 0;i < children.size();i++){
      children[i]->findByName(name);  //  Рекурсивно  продвигаемся  ниже  по
иерархии
   }
   if(_foundByNameCount == 1){ // Если во всей ветке только один объект с
заданным названием, возвращаем его
      return _foundChild;
   }else{
      return nullptr;
   }
}
Base* Base::findOnTreeByName(std::string name){ // Поиск объекта по имени во
всем дереве
   if(parent  !=  nullptr){  //  Если  есть  родитель,  продвигаемся  выше  по
иерархии
      return parent->findOnTreeByName(name);
   }
   _foundByNameCount = 0; // ОБнуляем счетчик и запускаем рекурсивный поиск
объекта
   return findByName(name);
}
std::string   getFirstBeforeDelimiter(std::string   s,   char   delimiter){
//Функция получения следующего объекта в пути
   std::string res = "";
   for(int i = 0; i < s.length(); i++){ // Ищем разделитель
      if(s[i]  !=  delimiter){  //  Если  текущий  символ  -  не  разделитель,
добавляем его к res
         res += s[i];
      }else{ // Иначе выходим из цикла и возвращаем res
         break;
      }
   }
   return res;
}
Base*  Base::findByCoords(std::string  coords){  //  Метод  поиска  объекта
покоординатам
```

```
   _foundByNameCount = 0;
   if(coords[0] == '/' && coords[1] == '/'){ // Если путь начинается с "//"
      return findOnTreeByName(coords.substr(2, coords.length() - 2)); //Вызов
метода поиска по всему дереву по имени. Имя объекта - путь без первых двух
символов
   }else if(coords[0] == '/'){ // Если путь абсолютный
      if(parent != nullptr){
         return parent->findByCoords(coords);
      }else{
         if(coords.length() == 1){
            return this;
         }else{
            return findByCoords(coords.substr(1, coords.length() - 1));
         }
      }
   }else if(coords[0] == '.'){ // Если путь начинается с точки
      if(coords.length()  == 1){  // Если путь равен точке, вернуть текущий
объект
         return this;
      }else{ // Иначе верунть результат поиска по ветке по имени объекта
         return findByName(coords.substr(1, coords.length() - 1));
      }
   }else{ // Если путь относительный
      std::string nowChild = getFirstBeforeDelimiter(coords, '/'); //Получить
имя первого объекта в пути
      if(nowChild.length() == coords.length()){ // Если это последний элемент
в пути
      for(int i = 0; i < children.size(); i++){ // Ищем этот элемент и, если
находим, возвращем его, иначе позвращаем нулевой указатель
         if(children[i]->getName() == nowChild){
            return children[i];
         }
      }
      return nullptr;
      }else{
         Base* found = nullptr;
         for(int i = 0; i < children.size(); i++){ // Ищем следующий элемент
среди детей текущего
            if(children[i]->getName() == nowChild){
               found = children[i];
            }
         }
         if(found != nullptr){  // Если нашли, вызываем рекурсивно метод, с
путем без найденного элемента (продвигаемся по пути)
            return    found->findByCoords(coords.substr(nowChild.length()    +
1,coords.length() - nowChild.length() - 1));
         }else{ // Иначе, возвращаем нулевой указатель
            return nullptr;
         }
      }
   }
}
void Base::deleteChildByName(std::string name){
   Base* temp = getChildByName(name);
   if(temp != nullptr){
      children.erase(std::find(children.begin(), children.end(), temp));
      delete temp;
```

```
    }
}
bool isMemberOfRoot(Base* root, Base* member){
    if(member->getParent() == nullptr){ // Если дошли до корня дерева, значит
искомый объект не является частью ветки от root
        return false;
    }
    if(member->getParent() == root){ // Если родитель равен root, значит
member принадлежит ветке от root, значит возвращаем ложь
        return true;
    }
    return isMemberOfRoot(root, member->getParent()); // Рекурсивнй
вызовметода, поднимаемся выше по дереву
}
bool Base::moveTo(Base* obj){ // Метод перемещения объекта к новому родителю
    if(parent == obj){ // Если пытаемся переместить к текущему
родителю,вернуть ложь
        return false;
    }
    if(isMemberOfRoot(this, obj)){ // Если новый родитель является частью
ветки от текущего объекта, вернуть ложь
        return false;
    }
    parent->children.erase(std::find(parent->children.begin(),parent-
>children.end(), this));
    parent = obj;
    obj->children.push_back(this); // Добавить текущий объект к детям нового
родитель
    return true;
}
void Base::addConnection(TYPE_SIGNAL signal, Base* base, TYPE_HANDLER
handler){
    o_sh* connection;
    for(int i = 0; i < connects.size();i++){
        if(connects[i]->p_signal == signal && connects[i]->p_base == base &&
connects[i]->p_handler == handler){
            return;
        }
    }
    connection = new o_sh();
    connection->p_signal = signal;
    connection->p_base = base;
    connection->p_handler = handler;
    connects.push_back(connection);
}
void Base::deleteConnection(TYPE_SIGNAL signal, Base* base, TYPE_HANDLER
handler){
    for(int i = 0; i < connects.size(); i++){
        if(connects[i]->p_signal == signal && connects[i]->p_base == base &&
connects[i]->p_handler == handler){
            o_sh* found = connects[i];
            connects.erase(connects.begin() + i);
            delete found;
            return;
        }
    }
}
```

```
void Base::emitSignal(TYPE_SIGNAL signal, std::string& command){
   if(_state != 0){
      (this->*signal)(command); // Вызов метода сигнала
      for(int i = 0; i < connects.size(); i++){
         if(connects[i]->p_signal == signal){ // Поиск подходящего сигнала
            TYPE_HANDLER handler = connects[i]->p_handler;
            Base* base = connects[i]->p_base;
            if(base->_state != 0){ // если объект включен
               (base->*handler)(command); // Вызов метода обработчика
            }
         }
      }
   }
}
std::string Base::get_path(){
   if(parent != nullptr){
      if(parent->parent != nullptr){
         return parent->get_path() + "/" + _name;
      }
      return parent->get_path() + _name;
   }
   return "/";
}
void Base::print(int offset){ // Вывод дерева иерархии объектов
   std::string s = "";
   for(int i = 0;i < offset;i++){
      s += "    ";
   }
   std::cout << s << _name << std::endl; // Вывод сдвига и названия объекта
   for(int i = 0;i < children.size();i++){
      children[i]->print(offset + 1); // Рекурсивный вызов метода вывода
увсех детей
   }
}
void Base::printWithState(int offset){ // Вывод дерева иерархии объектов со
статусом готовности
   std::string s = "";
   for(int i = 0;i < offset;i++){
      s += "    ";
   }
   std::cout << s << _name << (_state != 0 ? " is ready" : " is not ready")
<< std::endl; // Вывод сдвига, названия объекта и статуса готовности
   for(int i = 0;i < children.size();i++){
      children[i]->printWithState(offset + 1); // Рекурсивный вызов метод
вывода у всех детей
   }
}
```

Файл Base.h

Листинг 2 – Base.h

```
#ifndef __BASE__H
#define __BASE__H
#include <vector>
#include <string>
class Base;
#define SIGNAL_D(signal_f)(TYPE_SIGNAL)(&signal_f)
#define HANDLER_D(handler_f)(TYPE_HANDLER)(&handler_f)
typedef void (Base::*TYPE_SIGNAL)(std::string&);
typedef void (Base::*TYPE_HANDLER)(std::string);
class Base {
   struct o_sh{
      TYPE_SIGNAL p_signal;
      Base* p_base;
      TYPE_HANDLER p_handler;
   };
   private:
   Base* parent = nullptr;
   std::string _name;
   std::vector<Base*> children;
   std::vector<o_sh*> connects;
   int _state = 1;
   public:
   Base(Base* parent, std::string name = "Object");
   std::string getName();
   bool setName(std::string name);
   void setState(int state);
   Base* getParent();
   Base* getChildByName(std::string name);
   Base* findByName(std::string name);
   Base* findOnTreeByName(std::string name);
   Base* findByCoords(std::string coords);
   void deleteChildByName(std::string name);
   bool moveTo(Base* obj);
   int number = 1;
   void addConnection(TYPE_SIGNAL signal, Base* base, TYPE_HANDLER handler);
   void  deleteConnection(TYPE_SIGNAL  signal,  Base*  base,  TYPE_HANDLER
handler);
   void emitSignal(TYPE_SIGNAL signal, std::string& message);
   std::string get_path();
   void print(int offset = 0);
   void printWithState(int offset = 0);
   ~Base();
};
#endif
```

Файл cl_1.cpp

Листинг 3 – cl_1.cpp

```
#include "cl_1.h"
#include "iostream"
using namespace std;
cl_1::cl_1(Base* parent, string name): Base(parent,name){
}
void cl_1::signal_f(string& message){
   cout<<"Signal from "<<get_path()<< endl;
   message += " (class: 1)";
}
void cl_1::handler_f(string message){
   cout<< "Signal to "<<get_path()<<" Text: "<<message<<endl;
}
```

Файл cl_1.h

Листинг 4 – cl_1.h

```
#ifndef __CL_1__H
#define __CL_1__H
#include "Base.h"
using namespace std;
class cl_1 : public Base{
   public:
      cl_1(Base* base, string name);
      void signal_f(string& message);
      void handler_f(string message);

};


#endif
```

Файл cl_2.cpp

Листинг 5 – cl_2.cpp

```
#include "cl_2.h"
#include "iostream"
using namespace std;
cl_2::cl_2(Base *parent, string name): Base(parent,name)
```

```
{
   number=2;
}
void cl_2::signal_f(string& message){
   cout<<"Signal from "<<get_path()<< endl;
   message += " (class: 2)";
}
void cl_2::handler_f(string message){
   cout<< "Signal to "<<get_path()<<" Text: "<<message<<endl;
}
```

Файл cl_2.h

Листинг 6 – cl_2.h

```
#ifndef __CL_2__H
#define __CL_2__H
using namespace std;
#include "Base.h"
class cl_2: public Base
{
   public:
      cl_2(Base *parent, string name);
      void signal_f(string& message);
      void handler_f(string message);
};

#endif
```

Файл cl_3.cpp

Листинг 7 – cl_3.cpp

```
#include "cl_3.h"
#include "iostream"
using namespace std;

cl_3::cl_3(Base *parent, string name): Base(parent,name)
{
   number=3;
}
void cl_3::signal_f(string& message){
   cout<<"Signal from "<<get_path()<< endl;
   message += " (class: 3)";
}
void cl_3::handler_f(string message){
```

```
   cout<< "Signal to "<<get_path()<<" Text: "<<message<<endl;
}
```

Файл cl_3.h

Листинг 8 – cl_3.h

```
#ifndef __CL_3__H
#define __CL_3__H
using namespace std;
#include "Base.h"
class cl_3: public Base
{
   public:
      cl_3(Base *parent, string name);
      void signal_f(string& message);
      void handler_f(string message);

};


#endif
```

Файл cl_4.cpp

Листинг 9 – cl_4.cpp

```
#include "cl_4.h"
#include "iostream"
using namespace std;
cl_4::cl_4(Base *parent, string name): Base(parent,name)
{
   number=4;
}
void cl_4::signal_f(string& message){
   cout<<"Signal from "<<get_path()<< endl;
   message += " (class: 4)";
}
void cl_4::handler_f(string message){
   cout<< "Signal to "<<get_path()<<" Text: "<<message<<endl;
}
```

Файл cl_4.h

Листинг 10 – cl_4.h

```
#ifndef __CL_4__H
#define __CL_4__H
using namespace std;
#include "Base.h"
class cl_4: public Base
{
   public:
      cl_4(Base *parent, string name);
      void signal_f(string& message);
      void handler_f(string message);
};


#endif
```

Файл cl_5.cpp

Листинг 11 – cl_5.cpp

```
#include "cl_5.h"
using namespace std;
#include "iostream"
cl_5::cl_5(Base *parent, string name): Base(parent,name)
{
   number=5;
}
void cl_5::signal_f(string& message){
   cout<<"Signal from "<<get_path()<< endl;
   message += " (class: 5)";
}
void cl_5::handler_f(string message){
   cout<< "Signal to "<<get_path()<<" Text: "<<message<<endl;
}
```

Файл cl_5.h

Листинг 12 – cl_5.h

```
#ifndef __CL_5__H
#define __CL_5__H
```

```
using namespace std;
#include "Base.h"
class cl_5: public Base
{
   public:
      cl_5(Base *parent, string name);
      void signal_f(string& message);
      void handler_f(string message);
};


#endif
```

Файл cl_6.cpp

Листинг 13 – cl_6.cpp

```
#include "cl_6.h"
#include "iostream"
cl_6::cl_6(Base *parent, string name): Base(parent,name)
{
   number=6;
}
void cl_6::signal_f(string& message){
   cout<<"Signal from "<<get_path()<< endl;
   message += " (class: 6)";
}
void cl_6::handler_f(string message){
   cout<< "Signal to "<<get_path()<<" Text: "<<message<<endl;
}
```

Файл cl_6.h

Листинг 14 – cl_6.h

```
#ifndef __CL_6__H
#define __CL_6__H
using namespace std;
#include "Base.h"
class cl_6: public Base
{
   public:
      cl_6(Base *parent, string name);
      void signal_f(string& message);
      void handler_f(string message);
};
```

```
#endif
```

Файл cl_app.cpp

Листинг 15 – cl_app.cpp

```
#include "cl_app.h"
#include "cl_1.h"
#include "cl_2.h"
#include "cl_3.h"
#include "cl_4.h"
#include "cl_5.h"
#include "cl_6.h"
#include <iostream>
cl_app::cl_app(Base* root): Base(root){
}
void cl_app::build(){
   std::string root_name, parent_coords, child_name;
   Base *parent = this, // Родитель создаваемого объекта
   *child = this; // Создаваемый объект
   int clNumber;
   std::cin >> root_name;
   setName(root_name);
   while(true){
      std::cin >> parent_coords;
      if(parent_coords == "endtree") break; // Условие завершения цикла - имя
родительского объекта - endtree
      std::cin >> child_name >> clNumber;
      parent = findByCoords(parent_coords); // Ищем объект, ребенока которого
мы будем создавать
      if(parent   !=   nullptr   &&   parent->getChildByName(child_name)   ==
nullptr){// Если родитель существует и ребенка с таким именем еще нет
         switch(clNumber){  // Создаем объект класса в зависимости от числа
clParent
            case 2: {
               child = new cl_2(parent, child_name);
               break;
            }
            case 3: {
               child = new cl_3(parent, child_name);
               break;
            }
            case 4: {
               child = new cl_4(parent, child_name);
               break;
            }
            case 5: {
               child = new cl_5(parent, child_name);
               break;
            }
```

```
                case 6: {
                   child = new cl_6(parent, child_name);
                   break;
                }
             }
          }else if(parent == nullptr){ // Если не был наден родитель по заданному
пути
             std::cout << "Object tree" << std::endl;
             print();
             std::cout << "The head object " << parent_coords << " is not
found"<< std::endl;
             exit(1);
          }else{ // Если ребенок с заданным именем уже существует у текущего
родителя
             std::cout << parent_coords << "     Dubbing the names of subordinate
objects" << std::endl;
          }
       }
       Base* sender; // Отправитель сигнала
       Base* receiver; // Приемщик сигнала
       std::string sender_name, receiver_name;
       while(true){ // Связываем всех отправителей и приемщиков
          std::cin >> sender_name;
          if(sender_name == "end_of_connections"){
             break;
          }
          std::cin >> receiver_name;
          sender = findByCoords(sender_name);
          receiver = findByCoords(receiver_name);
          sender->addConnection(get_signal(sender-
>number),receiver,get_handler(receiver->number));
       }
    }
    int cl_app::exec(){
       std::cout << "Object tree" << std::endl;
       print();
       std::string command, message, coords;
       std::cin >> command;
       while(command != "END"){ // Считываем комманды, пока комманда не равна
END
          std::cin >> coords;
          Base* obj = findByCoords(coords);
          std::getline(std::cin, message);
          if(obj == nullptr){ // Если объект по координатам не был найден
             std::cout << "Object " << coords << " not found" << std::endl;
             std::cin >> command;
             continue;
          }
          if(command == "EMIT"){
             obj->emitSignal(get_signal(obj->number), message);
          }
          else if(command == "SET_CONDITION"){
             obj->setState(std::stoi(message));
          }else{
             message = message.substr(1); // Обрезаем пробел
             Base* c = findByCoords(message); // Ищем объект обработчики
             if(c == nullptr){
```

```
                std::cout << "Handler object " << coords << " not found"
<<std::endl;
                std::cin >> command;
                continue;
            }
            if(command == "SET_CONNECT"){
                obj->addConnection(get_signal(obj->number),c,get_handler(c-
>number));
            }
            else if(command == "DELETE_CONNECT"){
                obj->deleteConnection(get_signal(obj->number), c, get_handler(c-
>number));
            }
        }
        std::cin >> command;
    }
    return 0;
}
TYPE_SIGNAL cl_app::get_signal(int num){ // Метод получения сигнала
соответствующего класса
    switch(num){
        case 1:
            return SIGNAL_D(cl_1::signal_f);
            break;
        case 2:
            return SIGNAL_D(cl_2::signal_f);
            break;
        case 3:
            return SIGNAL_D(cl_3::signal_f);
            break;
        case 4:
            return SIGNAL_D(cl_4::signal_f);
            break;
        case 5:
            return SIGNAL_D(cl_5::signal_f);
            break;
        case 6:
            return SIGNAL_D(cl_6::signal_f);
            break;
        default:
            break;
    }
    return nullptr;
}
TYPE_HANDLER cl_app::get_handler(int num){ // Метод получения обработчика
соответствующего класса
    switch(num){
        case 1:
            return HANDLER_D(cl_1::handler_f);
            break;
        case 2:
            return HANDLER_D(cl_2::handler_f);
            break;
        case 3:
            return HANDLER_D(cl_3::handler_f);
            break;
        case 4:
```

```
            return HANDLER_D(cl_4::handler_f);
            break;
         case 5:
            return HANDLER_D(cl_5::handler_f);
            break;
         case 6:
            return HANDLER_D(cl_6::handler_f);
            break;
         default:
            break;
      }
      return nullptr;
   }
```

Файл cl_app.h

Листинг 16 – cl_app.h

```
#ifndef __CL_APP__H
#define __CL_APP__H
#include "Base.h"
using namespace std;
class cl_app: public Base
{
public:
   cl_app(Base *p);
   void build();
   int exec();
   TYPE_SIGNAL get_signal(int num);
   TYPE_HANDLER get_handler(int num);

};


#endif
```

Файл main.cpp

Листинг 17 – main.cpp

```
#include <stdlib.h>
#include <stdio.h>
#include "cl_app.h"

int main()
{
```

```
    cl_app app(nullptr);
    app.build();
    return app.exec();
}
```

ПРИЛОЖЕНИЕ 2. ТЕСТИРОВАНИЕ

Результат тестирования программы представлен в таблице 22.

Таблица 22 – Результат тестирования программы

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
appls_root / object_s1 3 / object_s2 2 /object_s2 object_s4 4 / object_s13 5 /object_s2 object_s6 6 /object_s1 object_s7 2 endtree /object_s2/object_s4 /object_s2/object_s6 /object_s2 /object_s1/object_s7 / /object_s2/object_s4 /object_s2/object_s4 / end_of_connections EMIT /object_s2/object_s4 Send message 1 EMIT /object_s2/object_s4 Send message 2 EMIT /object_s2/object_s4 Send message 3 EMIT /object_s1 Send message 4 END	Object tree appls_root object_s1 object_s7 object_s2 object_s4 object_s6 object_s13 Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: (class: 4) Signal to / Text: (class: 4) Object message not found Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: (class: 4) Signal to / Text: (class: 4) Object message not found Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: (class: 4) Signal to / Text: (class: 4) Object message not found Signal from /object_s1	Object tree appls_root object_s1 object_s7 object_s2 object_s4 object_s6 object_s13 Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: (class: 4) Signal to / Text: (class: 4) Object message not found Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: (class: 4) Signal to / Text: (class: 4) Object message not found Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: (class: 4) Signal to / Text: (class: 4) Object message not found Signal from /object_s1
	Object tree The head object is not found	Object tree The head object is not found
appls_root / object_s1 3 / object_s2 2	Object tree appls_root object_s1	Object tree appls_root object_s1

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
/object_s2 object_s4 4 / object_s13 5 /object_s2 object_s6 6 /object_s1 object_s7 2 endtree /object_s2/object_s4 /object_s2/object_s6 /object_s2 /object_s1/object_s7 / /object_s2/object_s4 /object_s2/object_s4 / end_of_connections EMIT /object_s2/object_s4 Send message 1 EMIT /object_s2/object_s4 Send message 2 EMIT /object_s2/object_s4 Send message 3 EMIT /object_s1 Send message 4 END	object_s7 object_s2 object_s4 object_s6 object_s13 Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: Send message 1 (class: 4) Signal to / Text: Send message 1 (class: 4) Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: Send message 2 (class: 4) Signal to / Text: Send message 2 (class: 4) Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: Send message 3 (class: 4) Signal to / Text: Send message 3 (class: 4) Signal from /object_s1	object_s7 object_s2 object_s4 object_s6 object_s13 Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: Send message 1 (class: 4) Signal to / Text: Send message 1 (class: 4) Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: Send message 2 (class: 4) Signal to / Text: Send message 2 (class: 4) Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: Send message 3 (class: 4) Signal to / Text: Send message 3 (class: 4) Signal from /object_s1
	Object tree The head object is not found	Object tree The head object is not found